Algorithm -Learning

Notes by Kundan Kumar(KK)

Table of Contents

INTRODUCTION

TIME/SPACE COMPLEXITY

DATA STRUCTURE
- ARRAYS
- LINKED LIST
- STACKS
- QUEUE
- BINARY SEARCH TREES
- BALANCED BINARY TREES
- RED-BLACK TREE
- HEAPS
- ASSOCIATIVE ARRAYS/DICTIONARIES
- TRIES

ALGORITHM

SORTING ALGORITHM

- LINKED LIST
- STACKS
- QUEUE
- BINARY SEARCH TREES
- BALANCED BINARY TREES
- RED-BLACK TREE

GRAPH ALGORITHM

- LINKED LIST
- STACKS
- QUEUE
- BINARY SEARCH TREES
- BALANCED BINARY TREES
- RED-BLACK TREE

INTRODUCTION :

- Data structures: to store data in an efficient way
- Why to use data structures?
- We often have the intuition if we want to make an algorithm fast, we have to optimize it
- Avoid nested for loops
- Make every calculation as fast as possible
- BUT algorithms can be boosted up by proper data structures
- Data structures make sure the running time will be better

Dijkstra algorithm

- Without a proper data structure (heap / priority queue) the running time would be quadratic // **O(N)**
- Priority queue approach makes sure the running time will be far better // **O(N*logN)**

Spanning trees

- We can boost up the algorithm with the help of priority queues as well
- So the running time of the algorithms are determined by the data structures we use !!!

ADT(Abstract data types)

- Basically this is the model (logical description) for a certain data structure
- It is like a supertype in programming (so an interface in Java)
- We just define what methods / functions the data structure will have, so we define the basic behavior.
- IMPORTANT: it is just the model, ADT does not specify the concrete implementation or programming language.
- basically what the user knows"
- For example: stack push() pop() peek()

Data Structure

- The concrete implementation, the actual representation of the data.
- The aim is to be able to store and retrieve data in an efficient manner.
- What we want: to be able to insert/find items in O (1) time complexity and to be able to retrieve items in O (1) as well.
- For example: arrays, linked lists, binary trees ...

Abstract data types	Data structures
Stack	array, linked list
Queue	array, linked list
Priority queue	heap
Dictionary / Hash-Map	array

COMPLEXITY :

Time Complexity

- What's the time complexity of the solution?
- Can you improve the time complexity of your solution?

Running time depends upon:

- Single vs Multi processor
- Read/Write Speed to memory
- 32-bit vs 64 bit
- Input
  The rate of growth of time with respect to input.

Model Machine

-Single processor
-32 bit
-Sequential execution
-1-unit time for arithmetical & Logical operations
-1 unit for assignment and return

Examples:

Sum (a, b)

```
{                    T-sum=one unit of return + one unit for addition-- Constant time
return a +b
    }
```

Sum of List (A, n)

	cost	no of times
{		
total=0	1(c1)	1
for I =0 to n-1	2 (c2)	n+1
total=total +Ai	2 (c2)	n
return total	1(c3)	1
}		

Total time=4n+4 ~ n

Time complexity: Depending on input size

T-sum=k	O (1)
T-sum of List= c. n +c'	O(n)
T-sum of Matrix=an^2+bn+c	O(n^2)

This is Big-oh notation or asymptotic notation.

Asymptotic Notations

Time Complexity analysis – asymptotic notations

Algorithm 1: T(n)=5n^2 +7
Algorithm 2: T(n)=17n^2+6n+8

1) N--->Infinity
2) No constant
   O- "big -oh" notation

O(g(n)) ={f(n): there exists constants C and no, f(n)<=cg(n), for n>=no

(read on video left)

Omega Notation:

Space Complexity

- Space complexity is a measure of how efficient your code is in terms of memory used.
- Space complexity analysis happens almost in the same way time complexity analysis happens.

For example, consider the following code:

```
vector<int> V;
for (int i = 0; i < N; i++) V. push_back(i);
```

The code snippet ends up creating a vector of size N. So, space complexity of the code is O(N).

Additional space / memory is measured in terms of the largest memory use by the program when it runs. That is to say, if you allocated O(N) memory, and later free it, that does not make the space complexity of your program **O (1).**

Relevance of Time Complexity:

Relevance of time complexity Lets assume we ask 2 interviewees A and B to write a program to detect if a number N >= 2 is prime.

A number is prime if it is divisible by exactly 2 distinct positive numbers 1 and the number itself.

A comes up with the following code:

```
i = 2
 while i < N
   if N is divisible by i
      N is not prime
   add 1 to i
```

B comes up with the following code :

```
i = 2
while i < square root of N
  if N is divisible by i
    N is not prime
  add 1 to i
```

For now, lets assume that both codes are correct.

Now, B claims that his code is much better as it takes much less time to check if N is prime.

Let's look into why that is the case.

Let's assume that the operation N is **divisible by i takes 1 ms**.

Let's look at few examples on time taken:

Example 1:

N = 1000033 (Prime number)
Time taken by A's program = 1 ms * number of divisions
= 1 ms * 1000033
= approximately 1000 seconds or 16.7 mins.

Time taken by B's program = 1ms * number of divisions
= 1ms * square root of 1000033
= approximately 1000ms = 1 second.

Example 2:

N = 1500450271 (Prime number)
Time taken by A's program = 1 ms * number of divisions
= 1 ms * 1500450271
= approximately 1000000 seconds or 11.5 days

Time taken by B's program = 1ms * number of divisions
= 1ms * square root of 1500450271
= approximately 40000ms = 40 seconds.

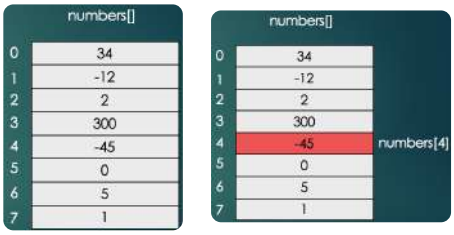
As you can see B's program is significantly faster even though both methods of solving the problem are correct.

- This is where time complexity of programs comes in, which is a measure of how efficient (or quick) a program is for large inputs.
- In first case, time taken is directly proportional to N, whereas in second case it is directly proportional to square root of N. In later slides, we will look into how we can formalize it into time complexity notations.

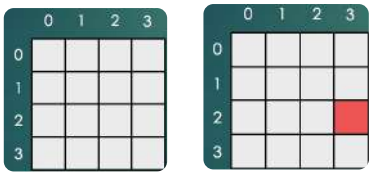
Data Structure:

ARRAYS

Arrays: data structures A collection of elements / values each identified by an array index or key!!!
- Index starts at zero
- Because of the indexes: random access is possible



Multidimensional arrays: it can prove to be very important in mathematical related computations (matrixes)



numbers[][] two dimensional array
First paramter: row index
Second parameter: column index

Arrays:
- Arrays are data structures in order to store items of the same type
- We use indices as keys !!!
- Arrays can have as many dimensions as we want: one or two dimensional arrays are quite popular
- For example: storing a matrix two dimensional array
- Dynamic array: when the size of the array is changing dynamically
- Applications: lookup tables / hashtables, heaps

-
Advantages :

-
- We can use random access because of the keys: getItem(int index) will return the value with the given key very fast // O(1)
- Very easy to implement and to use
- Very fast data structure
- We should use arrays in applications when we want to add items over and over again and we want to take items with given indexes!!! ~ it will be fast

-
-
-

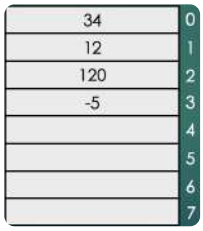
Disadvantages :

- We have to know the size of the array at compile-time: so it is not so dynamic data structure
- If it is full: we have to create a bigger array and have to copy the values one by one // reconstructing an array is O(N) operation
- It is not able to store items with different types

Arrays operation: add

We can keep adding values to the array as far as the array is not full

Step 1.add(34)
Step 2.add(12)
Step 3.add(120)
Step 4.add(-5)

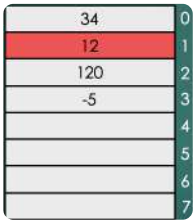


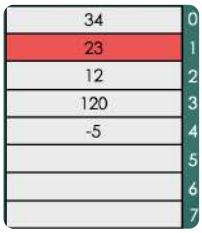
So: when adding new values to the List, we just have to insert it with the next index -----> very fast O(1) operation.

Arrays operation: insert item

We would like to insert a given value with a given index

insert(23,1)





So: it is a bit more problematic, sometime we have to shift lots of values in order to be able to insert the new one !!! ~ **O(N)** time complexity.

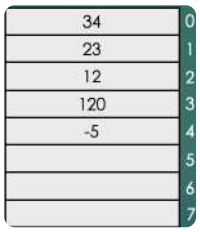
Add new item: **O (1)**
Insert item to a given index: **O(N)**

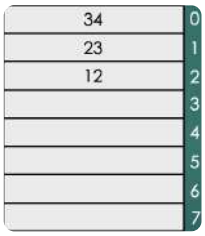
Arrays operation: remove items

We would like to remove the last item, it is very simple, just remove it // O (1) time complexity
Arrays operation: remove items with given index We would like to remove a value with a given index, it is not that simple, we may have to shift items // O(N) time complexity

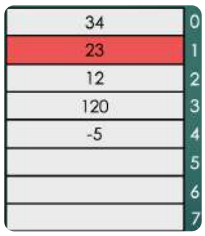
removeLast()

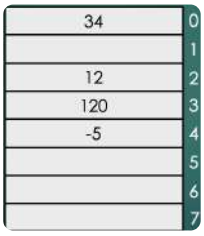
removeLast()

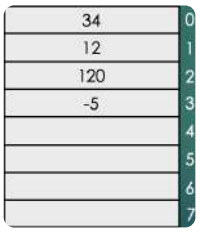




remove(1)







So: overall complexity will be linear O(N)

Removing last item: **O (1)**

Removing f.e. middle item: **O(N)**

Array:

- Stores data elements based on an sequential, most commonly 0 based, index.
- Based on tuple from set theory.
- They are one of the oldest, most commonly used data structures.

What you need to know:

Optimal for indexing; bad at searching, inserting, and deleting (except at the end).

- **Linear arrays**, or one dimensional arrays, are the most basic.
    - Are static in size, meaning that they are declared with a fixed size.
- **Dynamic arrays** are like one dimensional arrays, but have reserved space for additional elements.
    - If a dynamic array is full, it copies it's contents to a larger array.
- **Two dimensional arrays** have x and y indices like a grid or nested arrays.

Big O efficiency:

- Indexing: Linear array: O(1), Dynamic array: O(1)
- Search: Linear array: O(n), Dynamic array: O(n)
- Optimized Search: Linear array: O(log n), Dynamic array: O(log n)
- Insertion: Linear array: n/a Dynamic array: O(n)

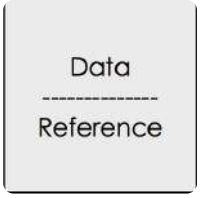
LINKED LIST:

- Linked lists are composed of nodes and references / pointers pointing from one node to the other!!!
    - The last reference is pointing to a NULL

-

A single node:

- contains data---> integer, double or custom object
- contains a reference pointing to the next node in the linked list




```
class Node {
    data
    Node nextNode
    ...
}
```

- Each node is composed of a data and a reference/link to the next node in the sequence
- Simple and very common data structure !!!
- They can be used to implement several other common data types: stacks, queues
- Simple linked lists by themselves do not allow random access to the data // so we can not use indexes ... getItem(int index) !!!
- Many basic operations such as obtaining the last node of the list or finding a node that contains a given data or locating the place where a new node should be inserted — require sequential scanning of most or all of the list elements

Advantages:

- Linked lists are dynamic data structures (arrays are not !!!)
- It can allocate the needed memory in run-time
- Very efficient if we want to manipulate the first elements
- EASY IMPLEMENTATION
- Can store items with different sizes: an array assumes every element to be exactly the same
- It's easier for a linked list to grow organically. An array's size needs to be known ahead of time, or re-created when it needs to grow

-
-
-

Disadvantages:

- Waste memory because of the references
- Nodes in a linked list must be read in order from the beginning as linked lists have sequential access (array items can be reached via indexes in **O (1)** time !!!)
- Difficulties arise in linked lists when it comes to reverse traversing. Singly linked lists are extremely difficult to navigate backwards,
- Solution: doubly linked lists easier to read, but memory is wasted in allocating space for a back pointer

Linked list operations : insertion

Inserting items at the beginning of the linked list: very simple, we just have to update the references----> **O(1) time** complexity

linkedList.insertAtStart(10);

linkedList.insertAtStart(4);

We just have to set the pointer to point to the next node.

linkedList.insertAtStart(-5);

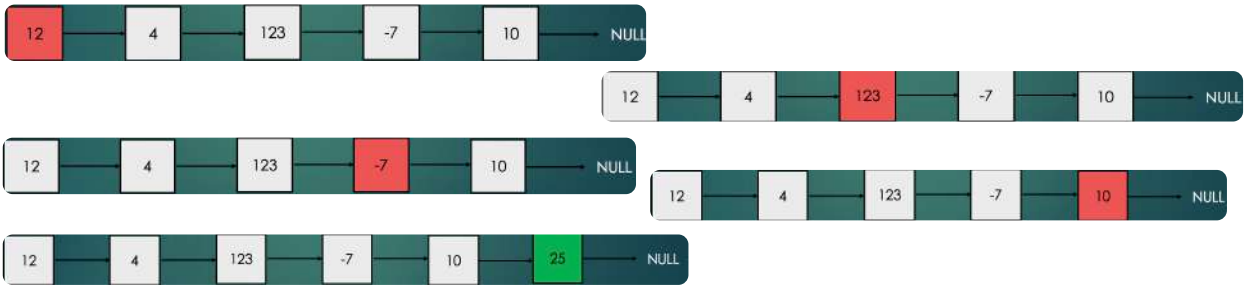
SO THIS OPERATION IS VERY FAST, THIS IS WHY WE LIKE LINKED LISTS !!!

- Inserting items at the end of the linked list: not that very simple, we have to traverse the whole linked list to find the last node.

How do we find the last node?

- We know the last node is pointing to a NULL !!! + we have to update the references when we get there O(N) time complexity

LinkedList.insertAtEnd(25);



1. We have to get to the last node: so is it pointing to a NULL? Yes, this is the last node .We have to update the references
2. Very important: updating the references again takes O(1) BUT we have to traverse the list itself and that what takes O(N) !!! // O(1) + O(N) = O(N)
3. Inserting items at the end of the linked list: not thatvery simple, we have to traverse the whole linked list to find the last node

   How do we find the last node? We know the last node is pointing to a NULL !!!

   + we have to update the references when we get there O(N) time complexity

Insert at the beginning	O(1)
Inserting at the end	O(N)

Linked List operations: remove

Remove item at the beginning of the List is always very fast: we do not have to search the item, we just have to update the references accordingly

O(1) time complexity.

linkedList.removeStart()

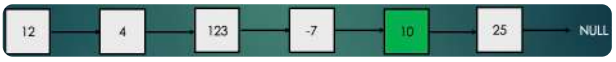
We just to remove it, very simple and fast operation !!!

O(N) time complexity

linkedList.remove(10)

Is it the item we are looking for? **NO, keep going**

Is it the item we are looking for? **YES,** we just have to update the references: the previous node should point to the next node



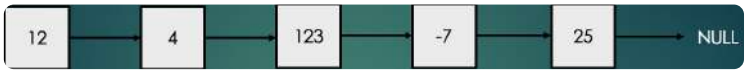
SO: we have managed to get rid of node with data 10, but we had to traverse the list

... O(N) time complexity

Remove items at the beginning: O(1)

Remove items at given positions: O(N) in the main

Problems with Linked Lists.



We can get from 4 to 25 because we just have to hop to the next nodes BUT we can not go from 25 to 4 because the references are in the opposite directions !!!

Solution: doubly linked List Node class has two references, one pointing to the next node, one pointing to the previous node

OK we can get from everywhere to everywhere BUT it is not so memory friendly, we have to store lots of references BUT there is no need to track the previous node during traversal !!! We can get from 4 to 25 because we just have to hop to the next nodes BUT we can not go from 25 to 4 because the references are in the opposite directions !!!

Linked Lists vs. Arrays

Search

1) Search operation yields the same result for both data structure
2) Array List search operation is pretty fast compared to the Linked List search operation
3) We can use random access with arrays: getItem(int index) which is O(1) time complexity
4) Linked List performance is O(N) time complexity
5) So the conclusion: ArrayList is better for this operation
6) **Why?**
7) Array List maintains index based system for its elements as it uses array data structure implicitly which makes it faster for searching an element in the list
8) On the other hand, Linked List requires the traversal through all the items for searching an element.

Deletion

- Linked List remove operation takes O(1) time if we remove items from the beginning and usually this is the case
- ArrayList: removing first element (so at the beginning) takes O(N) time, removing the last item takes O(1) times.
- But on average: we have to reconstruct the array when removing
- So the conclusion: Linked List is better for this operation
  - **Why**?
    - Linked List basically operates with pointers: removal only requires change in the pointer location which can be done very fast.

Memory-management

- Arrays don't need any extra memory
- LinkedLists on the other hand do need extra memory because of the references / pointers
- So in this aspect: arrays are better, they are memory friendly !!!

	LinkedList	Arrays
Search	O(N)	O(1)
Insert at the start	O(1)	O(N)
Insert at the end	O(N)	O(1)
Waste space	O(N)	0

Linked List Definition

- Stores data with **nodes** that point to other nodes.
- Nodes, at its most basic it has one datum and one reference (another node).
- A linked list *chains* nodes together by pointing one node's reference towards another node.

What you need to know:

Designed to optimize insertion and deletion, slow at indexing and searching.

- **Doubly linked list** has nodes that reference the previous node.
- **Circularly linked list** is simple linked list whose **tail**, the last node, references the **head**, the first node.
- **Stack**, commonly implemented with linked lists but can be made from arrays too.
  - Stacks are **last in, first out** (LIFO) data structures.
  - Made with a linked list by having the head be the only place for insertion and removal.
- **Queues**, too can be implemented with a linked list or an array.
  - Queues are a **first in, first out** (FIFO) data structure.
  - Made with a doubly linked list that only removes from head and adds to tail.

Big O efficiency:

Indexing: Linked Lists: O(n)

- Search: Linked Lists: O(n)
- Optimized Search: Linked Lists: O(n)
- Insertion: Linked Lists: O (1)

STACKS & QUEUE

Providing oversized data input to a program that does not check the length of input -> such a program may copy the data in its entirety to a location on the stack, and in so doing it may change the return addresses for procedures that have called it. An attacker can experiment to find a specific type of data that can be provided to such a program such that the return address of the current procedure is reset to point to an area within the stack itself (and within the data provided by the attacker), which in turn contains instructions that carry out unauthorized operations.

- buffer overflow attack
- If an oversized data item is moved to a stack location that is not large enough to contain it -> return information for procedure calls may be corrupted, causing the program to fail !!!

Recursion & Stack:

- Usually: we can avoid using stacks with recursive algorithms.
- BUT the operating system will use a stack in the background !!!

STACKS

- It is an abstract data type (interface)
- Basic operations: pop(), push() and peek()
- **LIFO** structure: last in first out

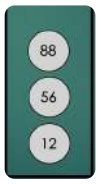
Stack Operation	Stack Contents	Return Value
s.is_empty()	[]	True
s.push(4)	[4]	
s.push('dog')	[4,'dog']	
s.peek()	[4,'dog']	'dog'
s.push(True)	[4,'dog',True]	
s.size()	[4,'dog',True]	3
s.is_empty()	[4,'dog',True]	False
s.push(8.4)	[4,'dog',True,8.4]	
s.pop()	[4,'dog',True]	8.4
s.pop()	[4,'dog']	True
s.size()	[4,'dog']	2

Table 3.1: Sample Stack Operations

- In most high level languages, a stack can be easily implemented either with arrays or linked lists
- A number of programming languages are stack-oriented, meaning they define most basic operations (adding two numbers, printing a character) as taking their arguments from the stack, and placing any return values back on the stack

Stack () creates a new stack that is empty. It needs no parameters and returns an empty stack.

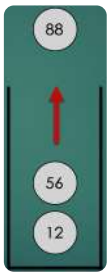
Push operation: put the given item to the top of the stack.Very simple operation, can be done in O(1)
stack.push(12);
stack.push(56);
stack.push(88);



• push(item) adds a new item to the top of the stack. It needs the item and returns nothing.

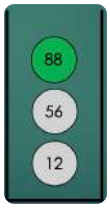
Pop operation: Take the last item we have inserted to the top of the stack (LIFO) .Very simple operation, can be done in O(1)
stack.pop();
stack.pop();

It removes the top item from the stack. It needs no parameters and returns the item. The stack is modified.



Peek operation: return the item from the top of the stack without removing it.
Very simple operation, can be done in O(1)

stack.peek();



• peek () returns the top item from the stack but does not remove it. It needs no parameters. The stack is not modified.

size (): returns the number of items on the stack. It needs no parameters and returns an integer.

is_empty () tests to see whether the stack is empty. It needs no parameters and returns a boolean value.

Applications:

-
  - In stack-oriented programming languages.
  - Graph algorithms: depth-first search can be implemented with stacks (or with recursion)
  - Finding Euler-cycles in a graph
  - Finding strongly connected components in a graph

Stack „call stack"

- Most important application of stacks: stack memory
- It is a special region of the memory (in the RAM)
- A call stack is an abstract data type that stores information about the active subroutines / methods / functions of a computer program
- The details are normally hidden and automatic in high-level programming languages
- Why is it good?
- It keeps track of the point to which each active subroutine should return control when it finishes executing
- Stores temporary variables created by each function
- Every time a function declares a new variable it is pushed onto the stack
- Every time a function exits all of the variables - pushed onto the stack by that function - are freed all of its variables are popped off of the stack // and lost forever !!!
- Local variables: they are on the stack, after function returns they are lost

- Stack memory is limited !!!

Heap memory

- The heap is a region of memory that is not managed automatically for you
- This is a large region of memory // unlike stack memory
- **C**: malloc() and calloc() function // with pointers
- **Java**: reference types and objects are on the heap
- We have to deallocate these memory chunks: because it is not managed automatically
- If not: memory leak !!!
- Slower because of the pointers

stack memory

- limited in size
- fast access
- local variables
- space is managed efficiently by CPU
- variables cannot be resized

heap memory

no size limits
slow access
objects
memory may be fragmented
variables can be resized // realloc()

Stack and recursion

- There are several situations when recursive methods are quite handy
- For example: DFS, traversing a binary search tree, looking for an item in a linked list ...
- What's happening in the background?
- All the recursive algorithms can be transformed into a simple method with stacks
- IMPORTANT: if we use recursion, the OS will use stacks anyways !!!

Depth-first search

Recursion

```
public void dfs(Vertex vertex) {

        vertex.setVisited(true);
        printf(vertex);

        for(Vertex v :
vertex.neighbours() ){
                if( !v.isVisited() ){
                        dfs(v);
                }
        }
}
```

iterative approach with stack

```
public void dfs(Vertex vertex) {

        Stack stack;
        stack.push(vertex);

        while( !stack.isEmpty() ){

                actual = stack.pop();

                for(Vertex v :
actual .neighbours() ){
                        if( !v.isVisited() ){

v.setVisited(true);
                                stack.push(v);
                        }
        }
}
```

Factorial: with recursion

```
public void factorial(int n) {

        if( n == 0 )
                return 1;

        return n * factorial(n-1);
}
```

This is the factorial function with
Recursive implementation

n! = n * (n-1) * ... * 2 * 1

For example: 4! = 4*3*2*1 = 24

What does it all have to do with stacks? The recursive function calls are pushed onto the stack until we bump into the base case

- we keep backtracking: we know the base case so we know the subsolutions
- if there are too many function calls to be pushed onto the stack: the stack may get full ... no more space left
- stack overflow !!!

Factorial: factorial(4)

```
public void factorial(int n) {

    if( n == 0 )
        return 1;

    return n * factorial(n-1);
}
```

int result = factorial(4)

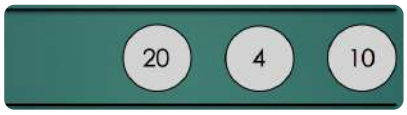
Result will be 4*3*2*1 = 24 !!!

QUEUE

- It is an abstract data type (interface)
- Basic operations: enqueue() and dequeue()
- FIFO: first in first out
- It can be implemented with dynamic arrays as well as with linked lists
- Important when implementing BFS algorithm for graphs

Enqueue operation: we just simply add the new item to the end of the queue

```
queue.enqueue(10);
queue.enqueue(4);
queue.enqueue(20);
```



Dequeue operation: we just simply remove the item starting at the beginning of the queue // FIFO structure

```
queue.dequeue();
queue.dequeue();
queue.dequeue();
```

Queue Operation	Queue Contents	Return Value
q.is_empty()	[]	True
q.enqueue(4)	[4]	
q.enqueue('dog')	['dog',4]	
q.enqueue(True)	[True,'dog',4]	
q.size()	[True,'dog',4]	3
q.is_empty()	[True,'dog',4]	False
q.enqueue(8.4)	[8.4,True,'dog',4]	
q.dequeue()	[8.4,True,'dog']	4
q.dequeue()	[8.4,True]	'dog'
q.size()	[8.4,True]	2

Table 3.5: Example Queue Operations

• **Queue ()** creates a new queue that is empty. It needs no parameters and returns an empty queue.
• **enqueue(item)** adds a new item to the rear of the queue. It needs the item and returns nothing.
• **dequeue ()** removes the front item from the queue. It needs no parameters and returns the item. The queue is modified.
• **is_empty ()** tests to see whether the queue is empty. It needs no parameters and returns a boolean value.
• **size ()** returns the number of items in the queue. It needs no parameters and returns an integer.

Application:

- When a resource is shared with several consumers (threads): we store them in a queue
- For example: CPU scheduling
- When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes
- For example: IO buffers
- Operational research applications or stochastic models relies heavily on queues !!!

Binary Search Trees

Sorted arrays

- Inserting a new item is quite slow // O(N)
- Searching is quite fast with binary search // O(logN)
- Removing an item is slow // O(N)

Linked lists

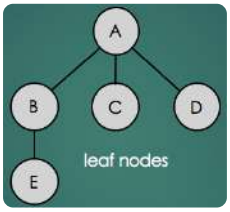
Inserting a new item is very fast // O(1)
Searching is sequental // O(N)
Removing an item is fast because of the references // O(1)

Binary search trees are going to make all of these operations quite fast, with O(log N) time complexity !!! ~ predictable

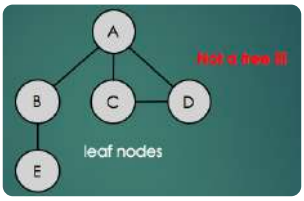
Trees

We have nodes with

the data and connection between the nodes // edges

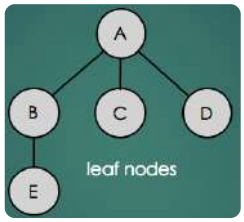


root node: we have a reference to this, all other nodes can be accessed via the root node



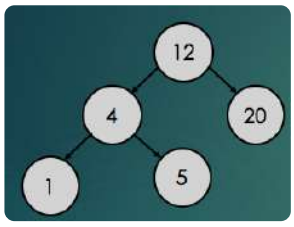
In a tree: there must be only a single path from the root node to any other nodes
in the tree
~ if there are several ways to get to a given node: it is not a tree !!!

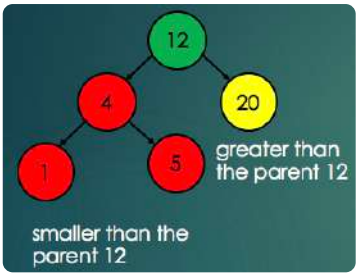
A node directly connected to another node child
The opposite -->parent node
Leaf nodes: with no children



Binary Search Trees

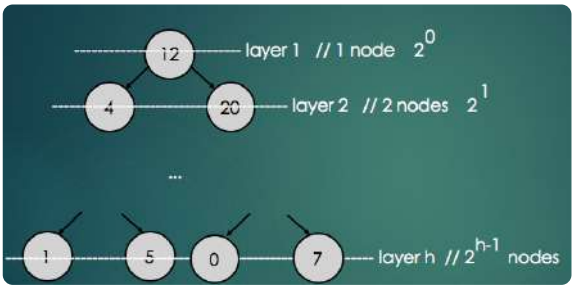
- every node can have at most two children: left and right child
- left child: smaller than the parent
- right child: greater than the parent





Why is it good On every decision we get rid of half of the data in which we are searching !!! // like binary search ~ O(logN) time complexity

Binary search trees: Height of a tree: the number of layers it contains



In general: h ~ O(logN) if this is true the tree is said to be balanced

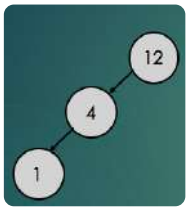
If it is not true the tree is unbalanced, which means it is asymmetric which is a PROBLEM !!!

Trees : height of a tree: the number of layers it has

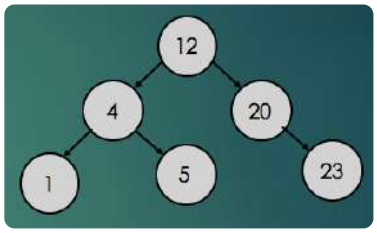
Height of the tree ‚h': the length of the path from the root to the deepest node in the tree

- we should keep the height of the tree at a minimum which is h=log n
- if the tree is unbalanced: the h=log n relation is no more valid and the operation's running time is no more logarithmic

Unbalanced Tree



Balanced Tree

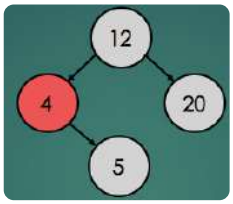


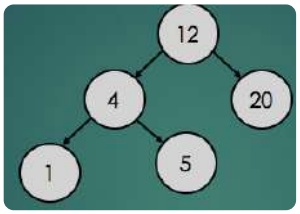
Binary search trees

- Binary search trees are data structures
- Keeps the keys in sorted order: so that lookup and other operations can use the principle of binary search !!!
- Each comparison allows the operations to skip over half of the tree, so that each lookup/insertion/deletion takes time proportional to the logarithm of the number of items stored in the tree.
- This is much better than the linear time O(N) required to find items by key in an unsorted array, but slower than the corresponding operations on hash tables.

Insertion: we start at the root node. If the data we want to insert is greater than the root node we go to the right, if it is smaller, we go to the left And so on ...

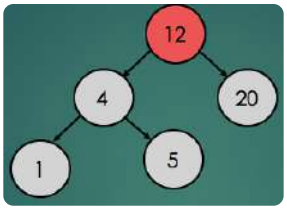
```
binarySearchTree.insert(12);
binarySearchTree.insert(4);
binarySearchTree.insert(5);
binarySearchTree.insert(20);
binarySearchTree.insert(1);
```

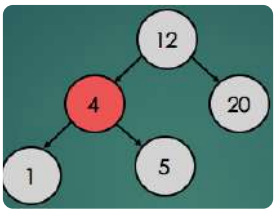


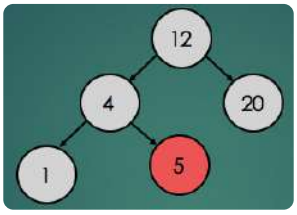


Search: we start at the root node. If the data we want to find is greater than the root node we go to the right, if it is smaller, we go to the left until we find it !!!

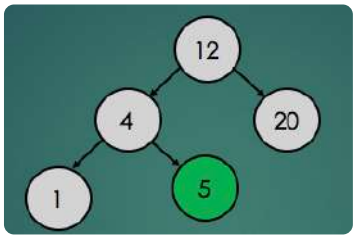
```
binarySearchTree.find(5);
```



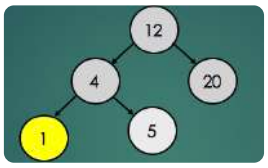




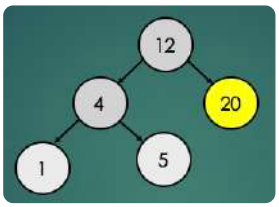
We have managed to find the item



On every decision: we discard half of the tree, so it is like binary search in a sorted array // O(logN)



We want to find the smallest node: we just have to go to the left as far as possible ... it will be the smallest !!!



We want to find the largest node: we just have to go to the right as far as possible ... it will be the largest !!!

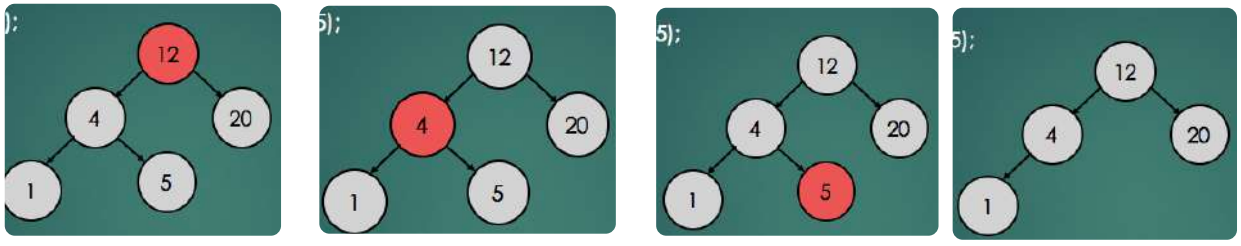
Delete: soft delete we do not remove the node from the BST we just mark that it has been removed
~ not so efficient solution

In the main three possible cases:

- The node we want to get rid of is a leaf node
- The node we want to get rid of has a single child
- The node we want to get rid of has 2 children

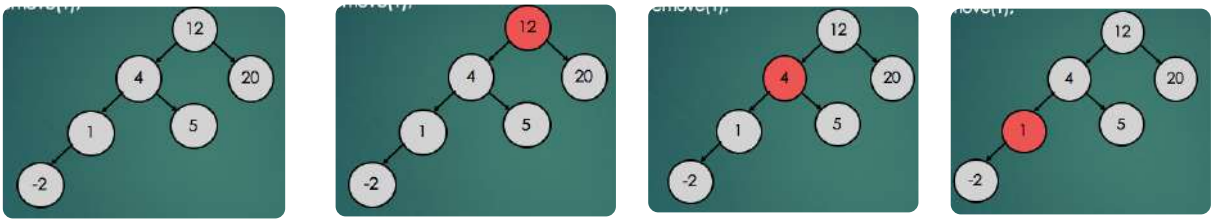
Delete: 1.) We want to get rid of a leaf node: very simple, we just have to remove it (set it to null whatever)

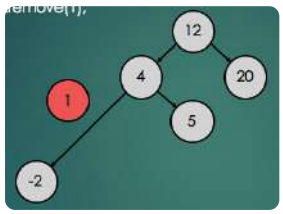
binarySearchTree.remove(5);

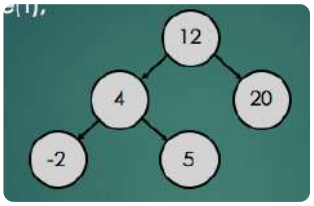


Complexity: we have to find the item itself + we have to delete it or set it to NULL
~ O(logN) find operation + O(1) deletion = **O(logN)** !!!

binarySearhTree.remove(1);





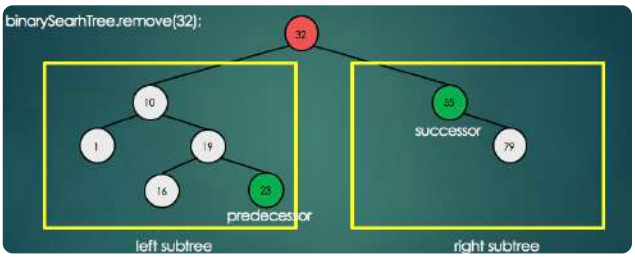


Complexity: first we have to find the item we want to get rid of and
we have to update the references
~ set parent's pointer point to it's grandchild directly

O(logN) find operation + O(1) update references = O(logN) !!!

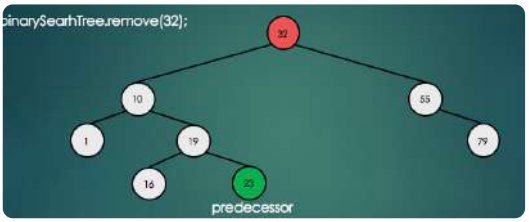
Delete: 3.) We want to get rid of a node that has two children

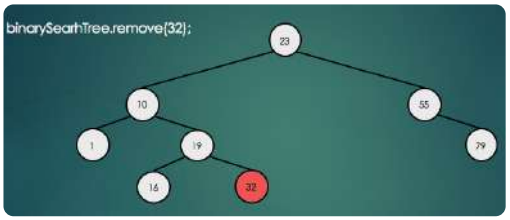
binarySearchTree.remove(32);



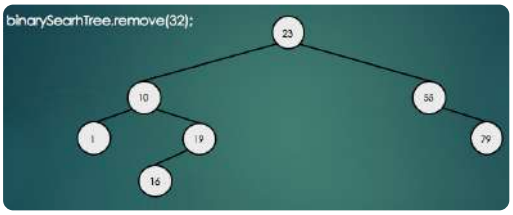
We have two options: we look for the largest item in the left subtree
OR the smallest item in the right subtree !!!

We look for the predecessor and swap the two nodes !!!

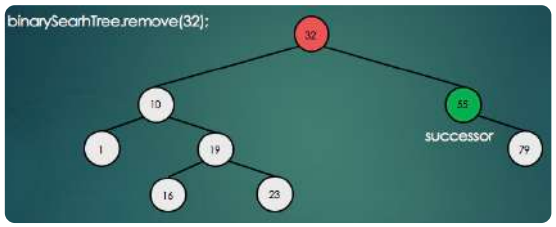


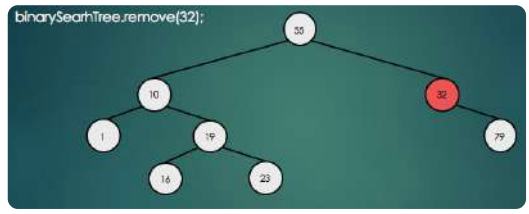


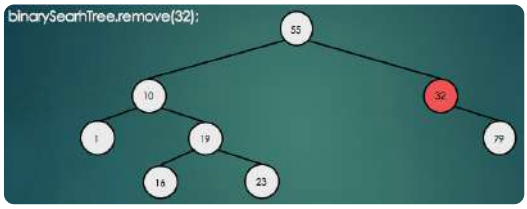
We end up at a case 1.) situation: we just have to set it to NULL

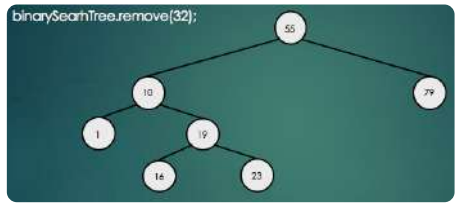


Another solution we look for the successor and swap the two nodes !!!







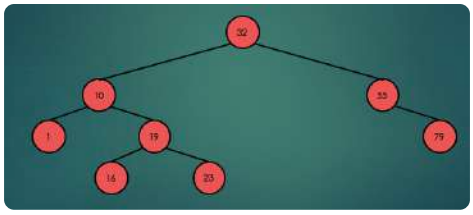


This becomes the Case 2.) situation, we just have to update the references
Complexity: **O(logN)**

Traversal: sometimes it is necessary to visit every node in the tree ,We can do it several ways

1. In-order traversal
2. Pre-order traversal
3. Post-order traversal

In-order traversal: we visit the left subtree + the root node + the right subtree recursively !!!



In-order traversal solution: 1 – 10 – 16 – 19 – 23 – 32 – 55 – 79
SO IT IS THE NUMERICAL ORDERING !!!

Pre-order traversal: we visit the root+ left subtree +the right subtree recursively !!!

Pre-order traversal: 32 – 10 – 1 – 19 – 16 – 23 – 55 – 79

Post-order traversal: we visit the left subtree+ right subtree +the root recursively !!!

What about the worst case scenarios?

- if the tree becomes unbalanced: the operations running times can be reduced to **O(N)** in the worst case
- thats why it is important to keep a tree as balanced as possible

	Average case	Worst case
Space	O(n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Search	O(log n)	O(n)

AVL(Balanced Tree)

Motivation

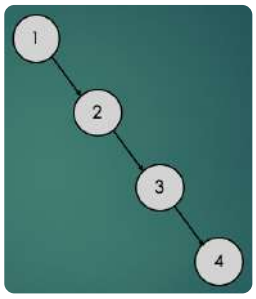
linked lists: quite easy to implement
Stores lots of pointers
O(N) search operation time complexity

- **binary search trees**: we came to to conclusion that **O(N)** searh complexity can be reduced to **O(logN)** time complexity
But if the tree is unbalanced : these operations will become slower and slower

- **balanced binary trees**: AVL trees or red-black trees
They are guaranteed to be balanced
Why is it good? **O(logN)** is guaranteed !!!

Construct a BST from a sorted array
[1,2,3,4]

[**1**,2,3,4]
[**1**,2,3,4]
[**1**,2,3,4]
[**1**,2,3,4]

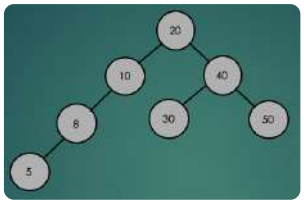


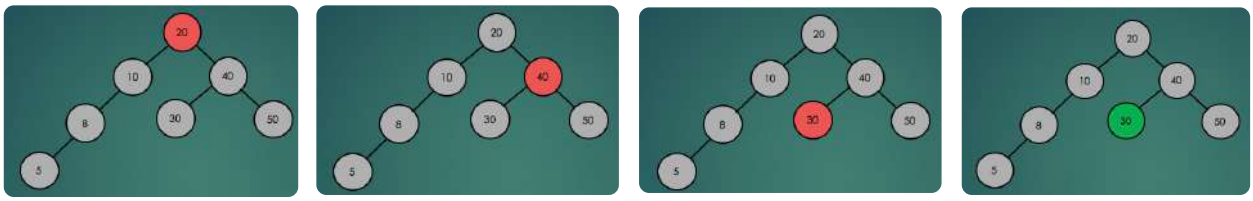
Conclusion: if we construct a binary search tree from a sorted array,
we end up with a linked list !!!
O(logN) reduced to **O(N)**

- The running time of BST operations depends on the height of the binary search tree: we should keep the tree balanced in order to get the best performance
- Thats why AVL trees came to be
- 1962: invented by two russian computer scientist
- In an AVL tree, the heights of the two child subtrees of any node differ by at most one
- Another solution to the problem is a red-black trees
- AVL trees are faster than red-black trees because they are more rigidly balanced BUT needs more work
- Operating systems relies heavily on these data structures !!!
- Most of the operations are the same as we have seen for binary search trees
- Every node can have at most 2 children: the leftChild is smaller, the rightChild is greater than the parent node.
- The insertion operation is the same BUT on every insertion we have to check whether the tree is unbalanced or not
- Deletion operation is the same

- Maximum / minimum finding operations are the same as well !!!

balancedTree.find(30);





findMin(); The minimum value in the tree: 5
findMax(); The maximum value in the tree: 50

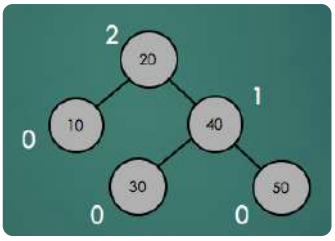
Binary search trees

	Average case	Worst case
Space	O(n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Search	O(log n)	O(n)

Balanced trees

	Average case	Worst case
Space	O(n)	O(n)
Insert	O(log n)	O(log n)
Delete	O(log n)	O(log n)
Search	O(log n)	O(log n)

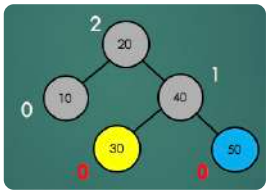
Height of a node: length of the longest path from it to a leaf
We can use recursion to calculate it:
height = max(leftChild.height(),rightChild.height())+1 !!!

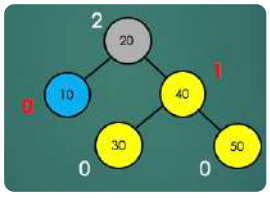


The leaf nodes have NULL children: we consider the height to be -1 for NULLs !!!

AVL algorithm uses heights of nodes, we want the heights as small as possible: we store the height parameters if it gets high, we fix it

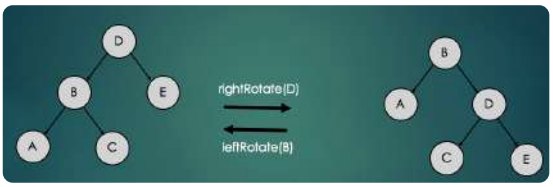
All subtrees height parameter does not differ more than 1 !!!





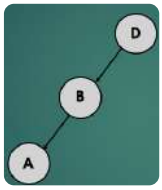
- AVL tree requires the heights of left and right child of every node to differ at most +1 or -1 !!!
- | height(leftSubtree) – height(rightSubtree) | < 1
- So for a balanced tree the height is in the range [-1;+1]
- We can maintain this property in O(logN) time which is quite fast !!!
- Insertion:
  - A simple BST insertion according to the keys
  - Fix the AVL property on each insertion from insertion upward
- There may be several violations of AVL property from the inserted node up to the root!!!
- We have to check them all

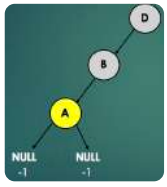
Rotations



We just have to update the references which can be done in **O(1)**
time complexity !!! (the in-order traversal is the same)

Rotations case I





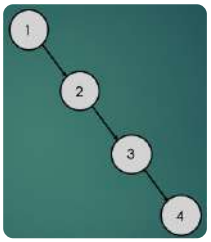
The difference of height parameters
is more than 1 !!! (actually it is 2)
~ so we make rotation to the right

Pseudocode case I

```
BEGIN rotateRight(Node node)

    Node tempLeftNode = node.getLeftNode()
    Node t = tempLeftNode.getRightNode()

    tempLeftNode.setRightNode(node)
    node.setLeftNode(t)

    node.updateheight()
    tempLeftNode.updateHeight()

END
```

Video:

RED-BLACK TREES



Construct a BST from a sorted array
[1,2,3,4]

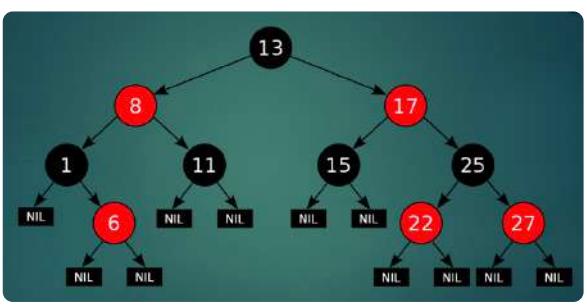
It is a linked list: **O(N)** !!!

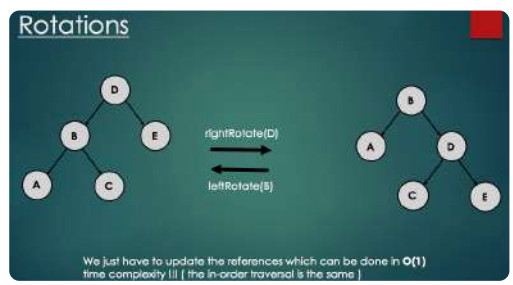
- The running time of BST operations depends on the height of the binary search tree: we should keep the tree balanced in order to get the best performance
- In an AVL tree, the heights of the two child subtrees of any node differ by at most one
- Another solution to the problem is a red-black trees
- AVL trees are faster than red-black trees because they are more rigidly balanced BUT needs more work
- Operating Systems relies heavily on these data structures !!!

	Average case	Worst case
Space	O(n)	O(n)
Insert	O(log n)	O(log n)
Delete	O(log n)	O(log n)
Search	O(log n)	O(log n)

Red-black properties:

- Each node is either red or black
- The root node is always **black**
- All leaves (NIL or NULL) are black
- Every red node must have two black child nodes and no other children it must have a black parent
- Every path from a given node to any of its descendant NIL/NULL nodes contains the same number of black nodes





- Every new node is red by default
- We keep inserting new node in the same way as we have seen for binary search trees (or AVL trees)
- On every insertion we have to check whether we have violated the red-black properties or not
- If we have violated the RB properties: we have to rebalance the tree
    - make rotations
    - OR just recolor the node

Case 1:

This problem is symmetric !!!

Watch video :

RED-BLACK TREES

- Linux kernels relies heavily on red-black tree data structure
- Insertion is fast -> because it is not rigidly balanced, we do not bother about making the tree as balanced as possible
- For an insert intensive tasks, use a Red-Black tree !!!
- Java: java.util.TreeMap , java.util.TreeSet
- C++ STL: map, multimap, multiset

AVL

-
- Rigidly balanced tree and hence provide faster look-ups
- For a look-up intensive task use an AVL tree
- Insertion/deletion is not so fast -> because we keep rebalancing the tree !!!

ASSOCIATIVE ARRAYS(ADT)

- Associative arrays / maps / dictionaries are abstract data types !!!
- Composed of a collection of key-value pairs where each key appears only once in the collection
- Most of the times we implement associative arrays with hashtables but binary search trees can be used as well

- The aim is to reach **O(1)** time complexity for most of the operations

Supported operations:

- Adding key-value pairs to the collection
- Removing key-value pairs from the collection
- Update existing key-value pairs
- Lookup of value associated with a given key

HASH TABLES / DICTIONARIES

Motivation :

Keys	Values
Goethe	Faust
Schiller	Don Carlos
Heidegger	Being and time

We would like to store authors and the titles of their novels So we have keys (authors) and values (titles)

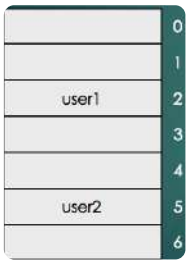
Keys	Values
daniel@gmail.com	User(„Daniel",24)
kevin@gmail.com	User(„Kevin",34)
adam@gmail.com	User(„Adam",56)

We would like to store users and the keys could be their email address. The aim would be to insert / retreive users according to their email address.

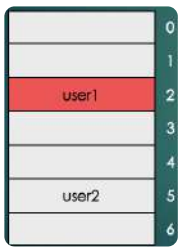
Motivation :

Arrays are just like that: if we know the index, the insert / retreive operations can be done in O(1) time

insert(2,user1)
insert(5,user2)



get(2)

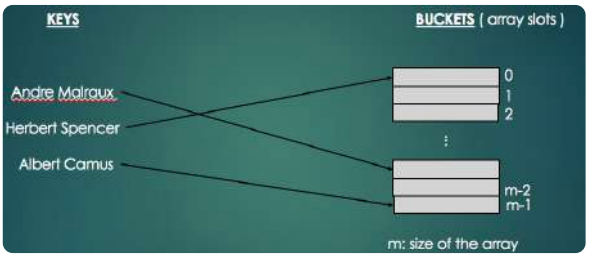


So arrays are going to solve our problem: the operations running time can be reduced to O(1)

PROBLEM: we must transform the keys into array indexes !!!
// this is why hash functions came to be

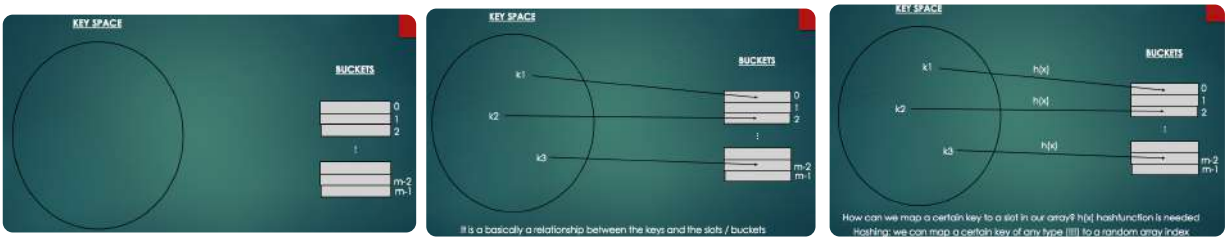
- Balanced BST we can achieve **O(logN)** time complexity for several operations including search
- Can we do better?
- Yes, maybe we can reach **O(1)** !!!
    - This is why hashtables came to be
- Array: if we know the index, the insertion and retrieval operations are very fast **O(1)** ... that is what we are after
- Here we want to search for a given item with a given key

We have key-value pairs



In general: we have ***n*** items to be stored + ***m*** buckets in which we can store items

Problem: keys are not always nonnegative integers. We have to do „prehashing" in order to map string keys to indexes of an array !!!



How can we map a certain key to a slot in our array? h(x) hash function is needed

Hashing: we can map a certain key of any type (!!!) to a random array index

- if we have integer keys we just have to use the modulo operator to transform the number into the range **[0,m-1]** ~ quite easy

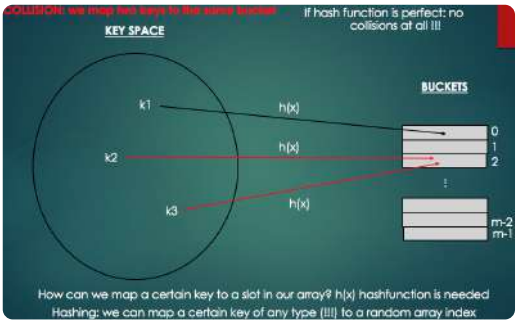
- if the keys are strings: we can have the ASCII values of the character and make some transformation in order to end up with an index to the array

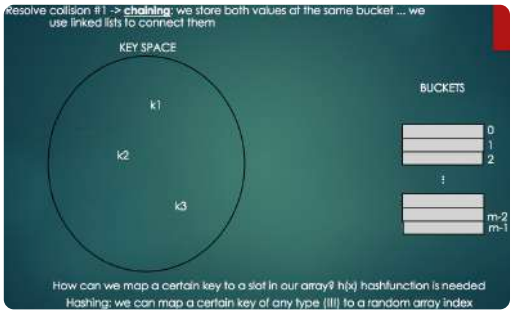
Hash function

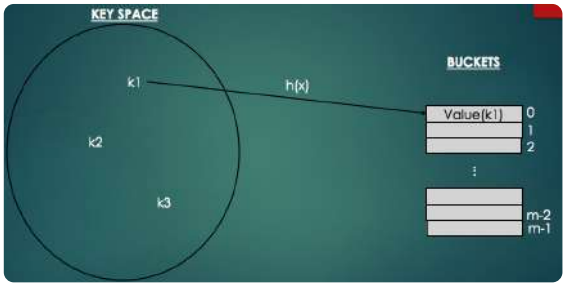
- Distribute the keys uniformly into buckets
- n: number of keys
- m: number of buckets // size of array
- h(x) = n % m (modulo operator)
    - We should use prime numbers both for the size of the array and in our hash function to make sure the distribution of the generated indexes will be uniform !!!
    - String keys: we could calculate the ASCII value for each character, add them up -> make % modulo

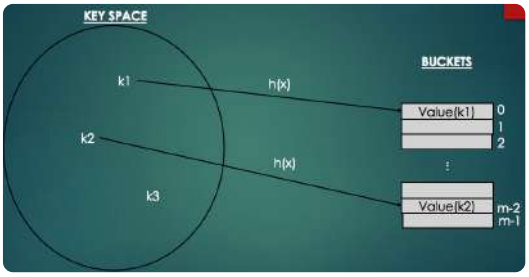
Collisions

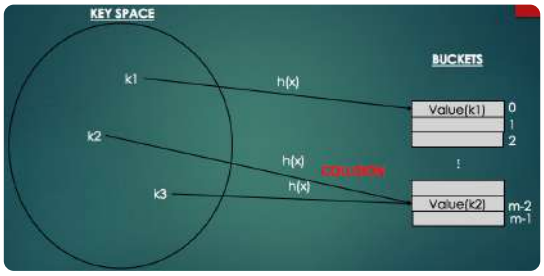
Resolve collision #2 -> **Chaining**:

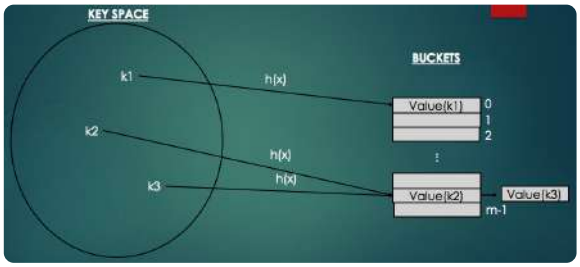




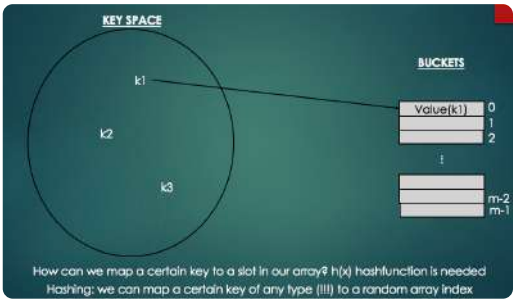


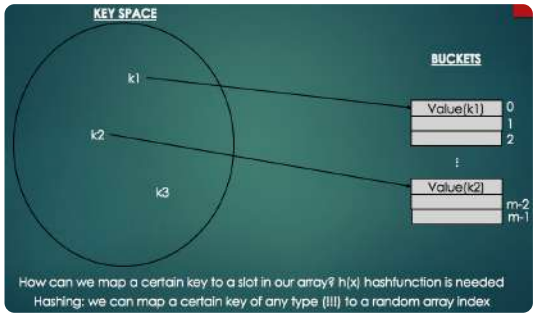


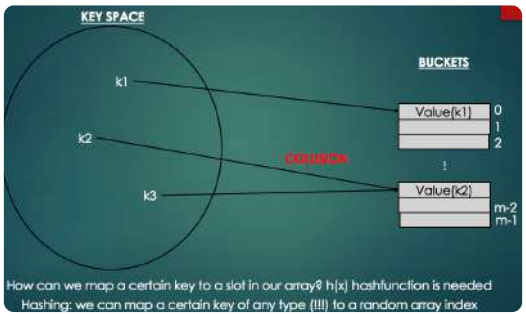


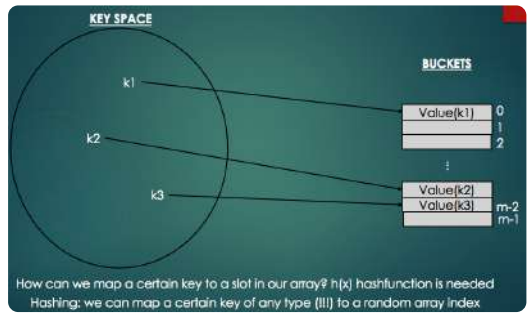


Resolve collision #2 -> **open addressing**: we generate a new index for the item









- Collision resolution with ***chaining***: we put multiple entries into the same slot with the help of a linked list
  - If there are many collisions: O(1) complexity gets worse !!!
  - It has an additional memory cost due to the references
- Collision resolution with **open addressing**: better solution
- If collision occurs, we find an empty slot instead
  - **Linear probing**: if a collision occures, we try the next slot … if there is a collision too we keep trying the next slot until we find an empty slot
  - **Quadratic probing**: we trying slots 1,2,4,8… units far away
  - **Rehashing**: we hash the result again in order to find an empty slot

	Average	Worst case
Space	O(n)	O(n)
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

Dynamic resizing

Load factor: number of entries divided by the number of slots / buckets

n/m =This is the load factor. It is 0 if the hash table is empty, it is 1 if the hashtable is full !!!

- if the load factor is approximately 1 it means it is nearly full: the performance will decrease, the operations will be slow
- if the load factor is approximately 0 it means it is nearly empty: there will be a lot of memory wasted

SO: dynamic resizing is needed sometimes !!!

- Performance depends on the load factor: what is the number of entries and number of buckets ratio
- Space-time tradeoff is important: the solution is to resize table, when its load factor exceeds given threshold
- **Java:** when the load factor is greater than 0.75, the hashmap will be resized automatically
- **Python:** the threashold is 2/3 ~ 0.66
- Hash values depend on table's size so hashes of entries are changed when resizing and algorithm can't just copy data from old storage to new one
- Resizing takes **O(n)** time to complete, where **n** is a number of entries in the table. This fact may make dynamic-sized hash tables inappropriate for real-time applications

Applications

- Databases: sometimes search trees, sometimes hashing is better
- Counting given word occurence in a particular document
- Storing data + lookup tables (password checks...)
- Lookup tables in huge networks (lookup for IP addresses)
- The hashing technique can be used for substring search

(Rabin-Karp algorithm)

PRIORITY QUEUES

- It is an abstract data type such as stack or queue
- BUT every item has an additional property: a priority value
- In a priority queue, an element with high priority is served before an element with lower priority
- Priority queues are usually implemented with heaps, but it can be implemented with self balancing trees as well
- Very similar to queues with some modification: when we would like to get the next item the highest priority element is retrieved first !!!

No FIFO structure here !!!

Integer values as priorities

Sometimes: we do not specify the priority // for example when implementing heap

- the value of an integer or double can be interpreted as a priority
- so we can omit the priority when inserting new integers or doubles
- the priority of 10 will be greater than that of 5 because 10 > 5 so there is no need to store the priority in another variable !!!

Operations

insertWithPriority(data, priority) // sometimes we do not specify the priority

This method will insert new item into the priority queue. We have to specify the data we want to insert and the priority associated with the given data

getHighestPriorityElement()

Returns the element with highest priority: we have to reconstruct the heap
Max heap: returns maximum element
Min heap: returns minimum element

peek()

Returns the element with highest priority: the structure of the heap does not change !!!

Sorting

- The concept of priority queues naturally suggest a sorting algorithm
- Insert all the elements to be sorted into a priority queue
- Sequentially remove them: it will be the sorted order !!!
- Why is it working?
  - We have been discussing that priority queues rely heavily on priorities
  - We take out items the one with highest prioirity will be returned
  - Result: sequency of decreasing priorities
  - This is the sorted order
  - For example: tree sort, heapsort

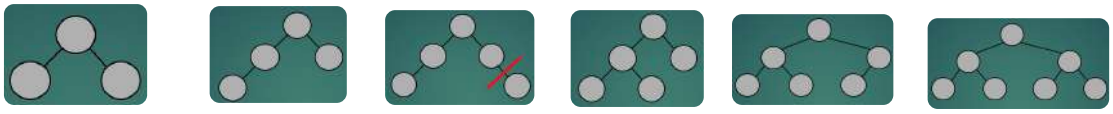
HEAPS

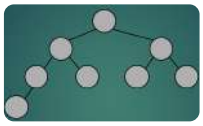
- It is baiscally a binary tree
- Two main binary heap types: min and max heap
- In a max heap, the keys of parent nodes are always greater than or equal to those of the children the highest key is in the root node.
- In a min heap, the keys of parent nodes are less than or equal to those of the children and the lowest key is in the root node
- It is complete: it cannot be unbalanced !!! We insert every new item to the next available place
- Applications: Dijkstra algorithm, Prims algorithm
- The heap is one maximally efficient implementation of a priority queue ADT
- It has nothing to do with the pool of memory from which dynamically allocated memory is allocated

Heap properties

1.) **Complete** -> we construct the heap from left to right across each row // of course the last row may not be completely full

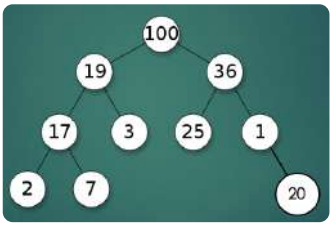
There is no missing node from left to right in a layer





Not a binary heap

This is not a heap: not complete



bec

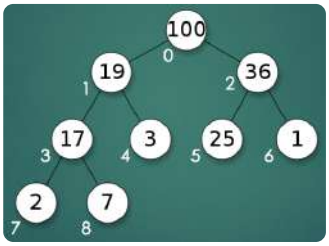
ause of this node

Heap properties

1.) **Complete** -> we construct the heap from left to right across each row // of course the last row may not be completely full
There is no mising node from left to right in a layer
2.) In a binary heap every node can have 2 children, left child and right child

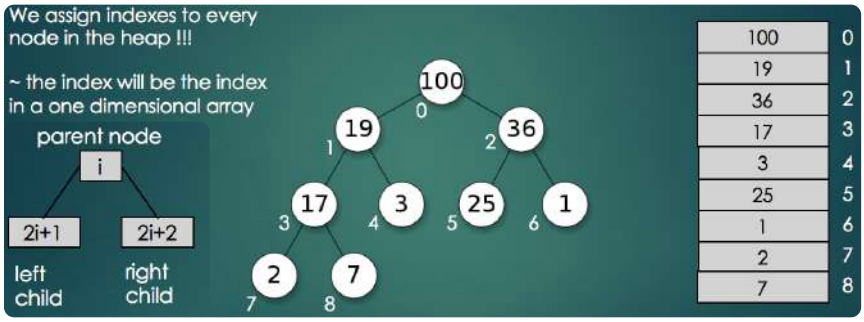
3.) Min heap ---> the parent is always smaller than the values of the children
Max heap --> the parent is always greater
So: the root node will be the smallest/ greatest value in the heap // O(1) access !!!

Binary heap: maximum heap

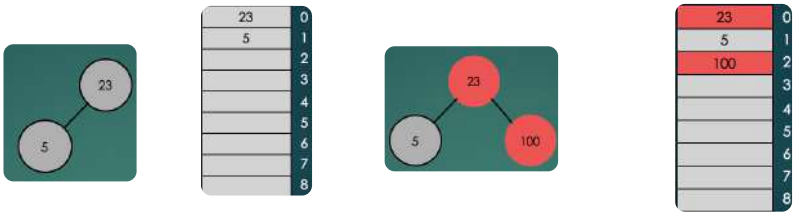


Represent heap as array

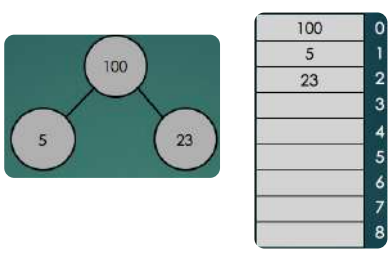
Accessing the root node:
array[0] !!!



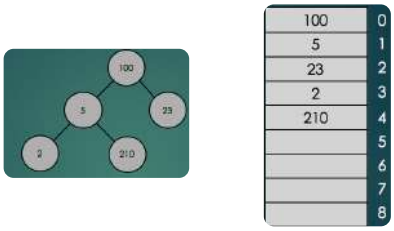
Building a heap: first we insert the data to the heap and we check whether the heap properties are met
~ if the heap properties are violated: we reconstruct the heap in order to make it a valid heap !!! "heapify process"

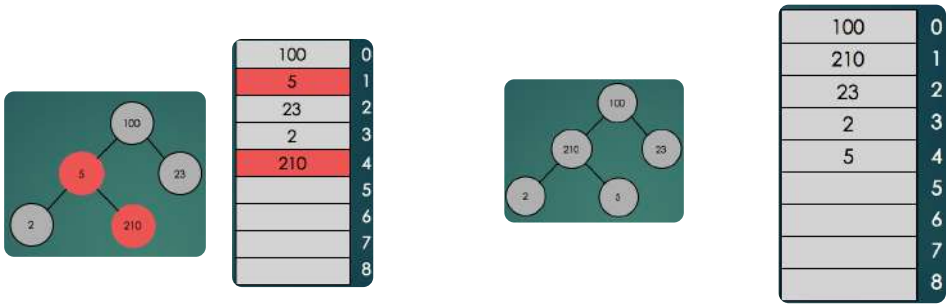


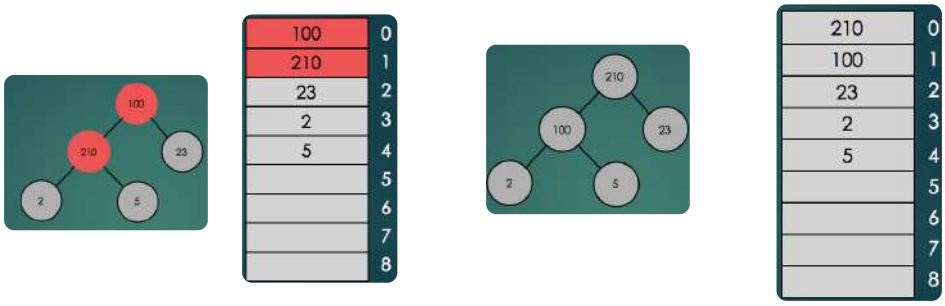
Now,



Insert: 2
Insert: 210





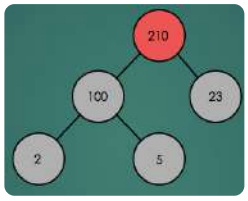


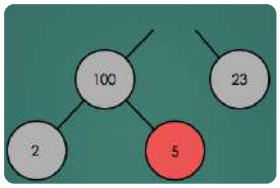
1. it is an O(N) process to construct a heap
2. OK we have to reconstruct it if the heap properties are violated but it takes O(logN) time
   O(N) + O(logN) = O(N)
3. inserting an item to the heap is just adding the data to the array with incremented index !!!

Remove operation

Deleting an item: we just get rid of the item we want to delete. OK, but there will be a „hole" in the tree. So we put the last item there, and make sure the heap properties are valid // with reconstructions !!!

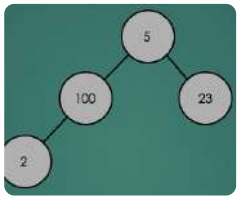
deleteNode(210);



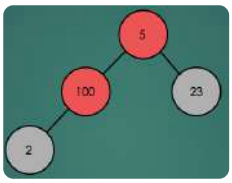


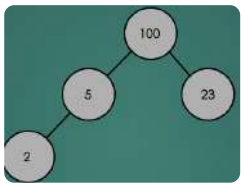
OK, we have to find the last item in the heap:

heapArray[lastIndex] ---> very easy to find it !!!



OK, we end up with a complete heap again, but we have to swap some items to make it valid !!!





So: we have managed to get rid of the root node and to make some reconstructions in order to end up with a valid heap again !!!

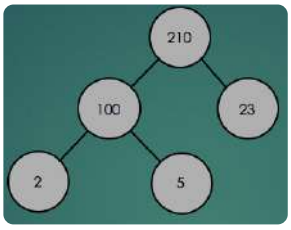
Operation: deleting the root node O(1) + reconstruction O(logN) = O(logN) !!!

Heapsort

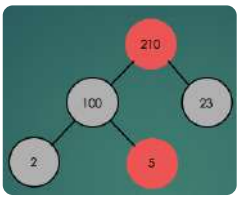
- Comparison-based sorting algorithm
- Use heap data structure rather than a linear-time search to find the maximum
- A bit slower in practice on most machines than a well-implemented quicksort, it has the advantage of a more favorable worst-case O(n log n) runtime
- It is an in-place algorithm, but it is not a stable sort
- DOES NOT NEED ADDITIONAL MEMORY

- Problem: first we have to construct the heap itself from the numbers we want to sort O(N) time complexity !!!

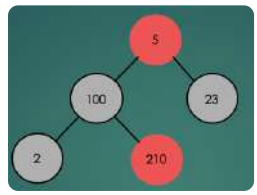
Keep swapping the root (because it is a heap,
we know for certain that the root is the item with highest priority)
with the last element + maintain heap properties !!!

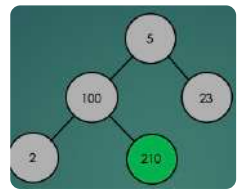


In this case, we are dealing with a max-heap, the root is the item with greatest value in the whole heap !!!

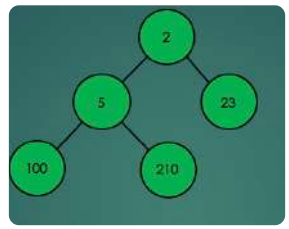


After swapping with the root:
- we consider the last item to be sorted: no longer part of the tree !!!
- check whether it is a valid heap or not





Sorted order: 210 , 100 , 23 , 5 , 2
We have managed to sort the elements !!!



Running time: we have to consider N items + have to make some swappings if necessary

O(N*logN)

Running time

Memory complexity: we have N items we want to store in the heap We have to allocate memory for an array with size N O(N) memory complexity

Memory complexity: we have N items we want to store in the heap .We have to allocate memory for an array with size N O(N) memory complexity

Find the minimum / maximum: O(1) very fast. Because in a heap the highest priority item is at the root node, it is easy heapArray[0] will be the item we are looking for

Insert new item: we can insert at the next available place, so increment the array, index and insert it O(1) fast BUT we have to make sure the heap properties are met ... it may take O(logN) time

O(log N) Why? Because a node has at most log N parents so at most log N swaps are needed

Remove item: we usually remove the root node
Removing it is quite fast: just delete it in O(1) time
BUT we have to make sure we met the heap properties O(logN) time to reconstruct the heap !!!

Operation	Time Complexity
Find minimum/maximum	O(1)
Remove min / max	O(log N)
Insert	O(log N)

Binomial heap

- Similar to a binary heap but also supports quick merging of two heaps
- It is important as an implementation of the mergeable heap abstract data type (meldable heap)
- Which is a priority queue basically + supporting merge operation
- A binomial heap is implemented as a collection of tree
- Insertion O(log n) time complexity can be reduced to O(1) constant time complexity with the help of binomial heaps

Fibonacci heap

- Faster than the classic binary heap
- Dijkstra's shortest path algorithm and Prim's spannig tree algorithm run faster if they rely on Fibonacci heap instead of binary heaps
- BUT very hard to implement efficiently so ususally does not worth the effort
- Unlike binary heaps, it can have several children: number of children are usually kept low
- We can achive O(1) insert operation instead of O(log n) !!!
- Every node has degree at most $O(\log n)$ and the size of a subtree rooted in a node of degree k is at least $F_k + 2$, where F_k is the k-th Fibonacci number

	Binary	Binomial	Fibonacci
Find min	O(1)	O(1)	O(1)
Delete min	O(log n)	O(log n)	O(log n)
Insert	O(log n)	O(1)	O(1)
Decrease key	O(log n)	O(log n)	O(1)
merge	-	O(log n)	O(1)

Sorting Algorithm

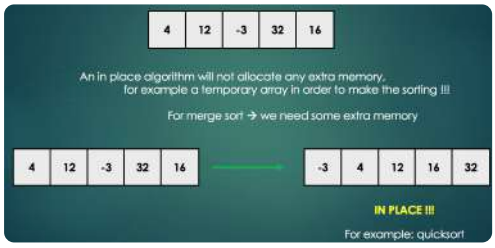
Sorting :

- A sorting algorithm is an algorithm that puts elements of an array in a certain order
- Numbers ---->numerical ordering !!!
- Strings, characters --->alphabetical ordering !!!
- **Comparison based** algorithms
    - ~ bubble sort, insertion sort, selection sort, merge sort, quicksort
- **Non-comparison based** sorting
    - ~ radix sort, bucket sort

Features

- Time complexity: **O(N)** or **O(N log N)** or **O(N)**
- In place: strictly an in-place sort needs only **O(1)** memory beyond the items being sorted . So an in place algorithm does not need any extra memory !!!
- Recursive: some sorting algorithms are implemented in a recursive manner the divide and conquer ones especially // merge sort and quicksort
- Stable: stable sorting algorithms maintain the relative order of records with equal values

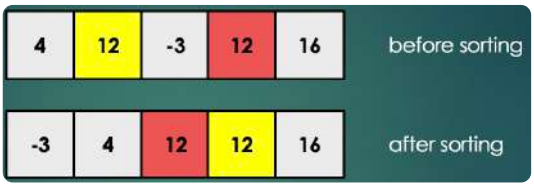
In place feature



- Sometimes we have some extra space when storing the numbers we want to sort not going to be in place !!!
-
- Why is it good to have algorithm that are in-place?

MEMORY EFFICIENT !!!

Stable algorithms



So the relative order of equal items remain the same

The red 12 is after the yellow 12 even after sorting !!!

Merge sort: stable

Quicksort: unstable

Lower bound

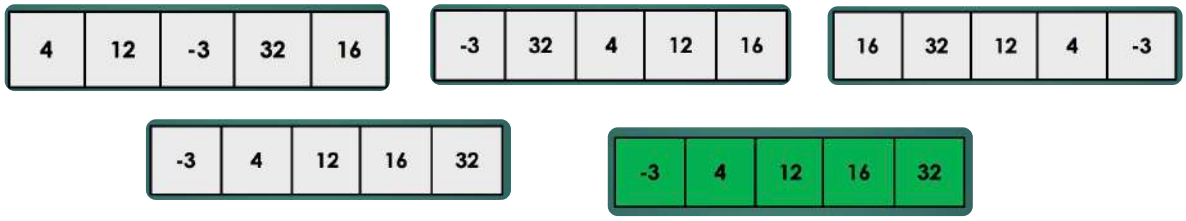
For sorting **N** items we have to make **log N!** comparisons With Stringling-formula it can be reduced to **N logN**

- so the **Ω(N logN)** time complexity is the lower bound for **comparison based** sorting algorithms
- ok but we can achieve **O(N)** running time as far as sorting is concernded, such as bucket sort or radix sort

THESE ARE NOT COMPARISON BASED ALGORITHMS !!!

Bogo sort

- Also known as **permutation sort** or **shotgun sort**
- A particularly ineffective sorting algorithm
- The algorithm keeps generating permutations of its input until it finds one that is sorted
- **O((n+1)!)** time complexity
- Two variant
    - **1.)** deterministic version that enumerates all permutations until it hits a sorted one
    - **2.)** randomized one: we randomly permutate the input until we find the solution // the sorted array



Why are we talking about the most inefficient sorting algorithm?

1.For classical computers it is inefficient

2.For **quantum computers O(1)** running time is guaranteed !!!

Because of quantum entanglement we can „search" for every possible combinations simultaneously

ADAPTIVE SORTING

- An adaptive algorithm is an algorithm that changes its behavior based on information available at runtime
- Adaptive sort it takes advantage of existing order in its input
- It benefits from local orders sometimes an unsorted array contains sequences that are sorted by default the algorithms will sort faster
- Most of the times: we just have to modify existing sorting algorithms in order to end up with an adaptive one

Adaptive algorithms

- Comparison based algorithms have optimal **O(N logN)** running time complexity
- Adaptive sort takes advantage of the existing order of the input to try to achieve better times: maybe **O(N)** could be reached
- The more presorted the input is, the faster it should be sorted
- **IMPORTANT**: nearly sorted sequences are common in practice !!!
- Heapsort, merge sort: not adaptive algorithms, do not take advantage of presorted sequences
- Shell sort: adaptive algorithm so performs better if the input is partially sorted

BUBBLE SORT

- Repeatedly steps through the list to be sorted, compares each pair of adjacent items and swaps them if they are in the wrong order
- It is too slow and impractical for most problems even when compared to insertion sort
- Bubble sort has worst-case and average complexity both **O(N)**
- Bubble sort is not a practical sorting algorithm
- It will not be efficient in the case of a reverse-ordered collection
- Stable sorting algorithm
- In place algorithm does not need any additional memory
- In computer graphics it is popular for its capability to detect a very small error (like swap of just two elements) in almost-sorted arrays and fix it with just linear complexity **O(N)**
- For example, it is used in a polygon filling algorithm, where bounding lines are sorted by their **x** coordinates at a specific scan line (a line parallel to **x** axis) and with incrementing **y** their order change (two elements are swapped) only at intersections of two lines

Pseudo Code:

bubbleSort(array)

```
    for i in range array.length-1   //We iterate through all the items in the array !!!

        for j in range array.length-1-i    //We keep considering fewer and fewer items, because
                                             on every iteration we consider one more item to be sorted !!!

            if array[j] > array[j+1]
                swap(array,j,j+1)
end
```

Watch Video:

SELECTION SORT

- Another **O(N)** running time sorting algorithm
- Selection sort is noted for its simplicity and it has performance advantages over more complicated algorithms
- Particularly useful where auxiliary memory is limited
- The algorithm divides the input list into <u>two parts</u>:
  - the subarray of items already sorted
  - and the subarray of items remaining to be sorted that occupy the rest of the array
-
- The algorithm proceeds by finding the smallest element in the unsorted subarray
- Exchange / swap it with the leftmost unsorted element putting it in sorted order
- Moving the subarray boundaries one element to the right
- It is an in place algorithm no need for extra memory
- Selection sort almost always outperforms bubble sort
- Not a stable sort does not preserve the order of keys with equal values
- Quite counter-intuitive: selection sort and insertion sort are both typically faster for small arrays // arrays with **10-20** items
- Usual optimization method recursive algorithms switch to insertion sort or selection sort for small subarrays
- Makes less writes than insertion sort this can be important if writes are significantly more expensive than reads,
- For example with **EEPROM** or flash memory where every write lessens the lifespan of the memory

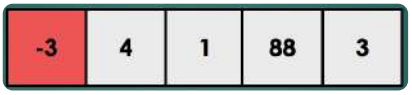
Pseudo Code:

```
selectionSort(array)

    for i in range array.length-1            //We have to consider all the items

        index = i

        for j from i+1 to array.length
            if array[j] < array[index]   //Basically we make simple linear search
                index = j                          for the minimum element !!!


            if index not i                  //If index = i it means the smallest item is index i so no need to swap
                swap(array, index, i)                  the number with itself

end
```

We find the minimum: for this we have to iterate through the whole array with O(N) time complexity ~ linear search

Minimum item: -3 swap it with the leftmost item
Smallest value: 1 we have to swap it with the leftmost item that has not been considered

Minimum item: 3 we have to swap it with the leftmost item that has not been sorted yet



Watch Video:

Sorting

Algorithm

BUBBLE SORT

- Repeatedely steps through the list to be sorted, compares each pair of adjacent items and swaps them if they are in the wrong order
- It is too slow and impractical for most problems even when compared to insertion sort
- Bubble sort has worst-case and average complexity both *O(N*N)*
- Bubble sort is not a practical sorting algorithm
- It will not be efficient in the case of a reverse-ordered collection !!!
- In computer graphics it is popular for its capability to detect a very small error (like swap of just two elements) in almost-sorted arrays and fix it with just linear complexity O(N). For example, it is used in a polygon filling algorithm, where bounding lines are sorted by their x coordinate at a specific scan line (a line parallel to x axis) and with incrementing y their order changes (two elements are swapped) only at intersections of two lines

- Stable algorithm like insertion sort

Watch Video:

HYBRID ALGORITHMS

- It combines more algorithms to solve a given problem
- It choses one algorithm depending on the data or switching between them over the course of the algorithm
- This is generally done to combine desired features of each, so that the overall algorithm is better than the individual components
- Important: hybrid algorithm does not refer to simply combining multiple algorithms to solve a different problem but only to combining algorithms that solve the same problem but differ in other characteristics // such as performance
- The technique can be used when sorting
    - Heapsort it has an advantage of a guaranteed running time **O(N logN)**
    - Quicksort optimal implementations are outperform both mergesort and heapsort
    - **BUT** quicksort can have quadratic running time when we keep choosing „bad" pivots
    - Solution: let's combine the two algorithms

Introsort

- Also known as introspective sort
- It is a hybrid sorting algorithm that provides both fast avarage performance and optimal worst-case performance
- It begins with quicksort and switches to heapsort when quicksort becomes too slow
- **INTROSORT = QUICKSORT + HEAPSORT**

Hybrid algorithms

- Insertion sort: very efficient on small data (**5 - 10** elements)

- Mergesort / quicksort: asymptotically optimal on large datasets, but the overhead becomes significant if applying them to small datasets
- Solution: let's combine the two algorithms
- Highly optimized hybrid algorithm: timsort
- **TIMSORT = INSERTION SORT + MERGESORT**

Timsort

- Combines mergesort and insertion sort
- It is a stable sorting algorithm
- It was implemented by Tim Peters in 2002 for use in the Python programming language
- Best case running time: **O(N)**
- Worst case running time: **O(N logN)**
- Worst case space complexity: **O(N)**

Insertion sort

- It is a simple sorting algorithm that builds the final sorted array one item at a time
- It has quadratic running time **O(N)**
- On large datasets it is very inefficient but on arrays with **10-20** items it is quite good
- Simple implementation !!!
- It is more efficient than other quadratic running time sorting procedures such as bubble sort or selection sort
- Adaptive algorithm speeds up when array is already substantially sorted
- Stable sort -->preserve the order of the items with equal keys
- In-place algorithm does not need any additional memory
- It is an online algorithm--->it can sort an array as it receives it for example downloading data from web
- Hybrid algorithms uses insertion sort if the subarray is small enough
- Insertion sort is faster for small subarrays than quicksort !!!
- Variant of insertion sort is shell sort
- Sometimes selection sort is better: they are very similar algorithms
- Insertion sort requires more writes because the inner loop can require shifting large sections of the sorted portion of the array
- In general, insertion sort will write to the array **O(n2)** times while selection sort will write only **O(n)** times
- For this reason selection sort may be preferable in cases where writing to memory is significantly more expensive than reading for example flash memory

Pseudocode

```
insertionSort(array)

    for i=1 to length(array)                //We iterate through the array sequentially !!!

        j = i

        while j > 0 and array[j-1] > array[j]    ///While the previous item is greater than the given
            swap( array, j, j-1 )                    one, we keep swapping them !!! ~ thats why there
            j = j – 1                                are so many shifts in insertion sort
        end
    end
end
```

Watch Video::

Mergesort

- Mergesort is a divide and conquer algorithm that was invented by John von Neumann in 1945
- Comparison based algorithm with running time complexity **O(N logN)**

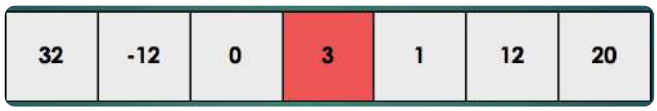
- It is a stable sorting algorithm
- Not an in-place algorithm !!!
- Although heapsort has the same time bounds as merge sort heapsort requires only **Θ(1)** auxiliary space instead of merge sort's **Θ(*n*)**
- Efficient quicksort implementations generally outperforms mergesort
- Merge sort is often the best choice for sorting a linked list: in this situation it is relatively easy to implement a merge sort in such a way that it requires only **Θ(1)** extra space

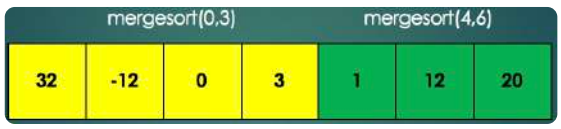
- divide the array into two subarrays recursively
- sort these subarrays recursively with mergesort again
- if there is only a single item left in the subarray we consider it to be sorted by definition
- merge the subarrays to get the final sorted array

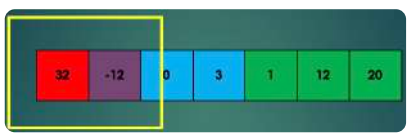
	Quicksort	Mergesort
In place	Yes	No
Stable	No	Yes
Time complexity	Quadratic sometimes	O(N logN)

Mergesort „divide"

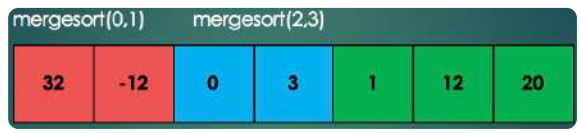
We call the sort method over and over again: mergesort(0,6).
Generate a middle index in order to partition the array: middleIndex = (low+high) / 2





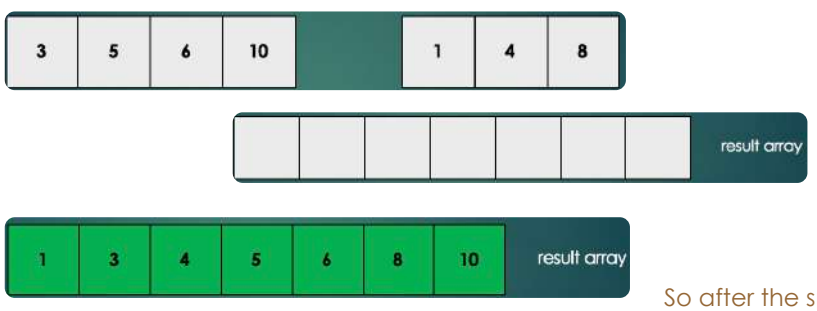


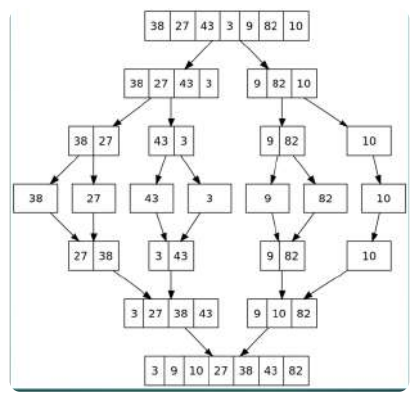
After several recursive method calls: we end with single items,



we consider them sorted by default
~ so we keep merging these already sorted items

Mergesort „Conquer"





So after the split operations: we have several distinct arrays that are already sorted: we have to merge these arrays into a single one

We start at the beginning of the subarrays: we keep comparing them, we insert the smaller into the result array

VERY IMPORTANT: we have to iterate through the left and right array if there are some more items left in this case the **10** in the left subarray

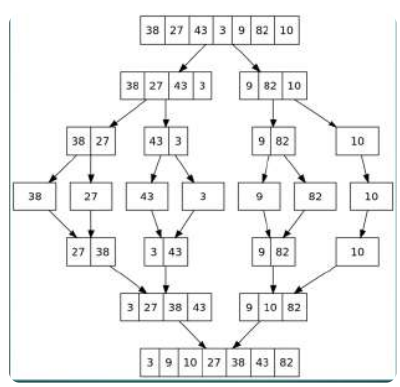
Watch Video:

Pseudocode

-
mergeSort(low,high)

```
    if (low >= high) return     //Base case for recursive method calls, in this situation the sort is over
    middle = (low + high) / 2  //We look for the middle index to partition the array into two equal subarrays

    mergeSort(low, middle)    //Call the mergesort method recursively on the left subarray
                              IMPORTANT: because of middle, there will be always more items in the left subarray !!!

    mergeSort(middle + 1, high)   //Call mergesort recursively on the right subarray IMPORTANT: because
                                    middle+1 there areat most as many items in the right subarray

    merge(low, middle, high)   //The conquer part of the algorithm, we keep merging together the subarrays
        end

    merge(low, middle, high) {

    for i=low to high              //Create a temporary array: size is equal to the size of the input Thats why
    tempArray[i] = nums[i];          mergesort has O(N) memory complexity  we have to use an other array

     i = low;                                  // Create the variables to be able to track the indexes
     j = middle + 1;
     k = low;

    while i <= middle &&  j <= high    //While we have items in the left and right subarrays

            if  tempArray[i] <= tempArray[j]   //The left subarray item is smaller: we put it to its right / sorted
             nums[k] = tempArray[i]              position in the array.Note: we keep merging the items from
                    i++                            temp array to the original nums array

          else                          //The right subarray item is smaller: we put it to its right / sorted position in
        nums[k] = tempArray[j];     the array.Note: we keep merging the items from temp  array to the
                    j++                        original nums array
          k++;

      while i <= middle            //Sometimes we have items left in the left Subarray: so copy it to the final
    nums[k] = tempArray[i];     nums array ~ it is in sorted order so just copy them. IMPORTANT: because we
        k++;                    partition our array in a way that the left subarray contains more items  we
        i++;                          just have to consider the left subarray ~ so we do not have to copy items
                                                                  from right subarray too !!
end
```

```
mergeSort(low,high)
    if (low >= high) return
    middle = (low + high) / 2
    mergeSort(low, middle)
    mergeSort(middle + 1, high)
    merge(low, middle, high)
end
```



Comparison based sorting

What does comparison based sorting mean?

if nums[i] > nums[j]
swap(i,j)

We keep comparing items (strings, characters, doubles ...)
~ keep making decisions according to these comparisons

Result: we have to make at least **log n!** comparisons to sort an array
Stirling formula yields: **Ω(N log N)**
So this is a lower bound, we are not able to do any better!!!

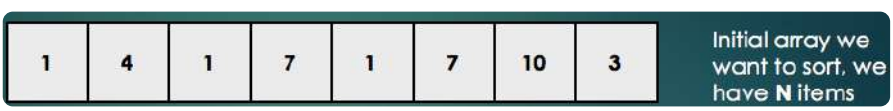
Non-Comparison based sorting

Can we do better? **YES**, the solution is not to use comparisons
There are simpler algorithms that can sort a list using partial information about the keys
For example: radix sort, bucket sort

Counting sort

-

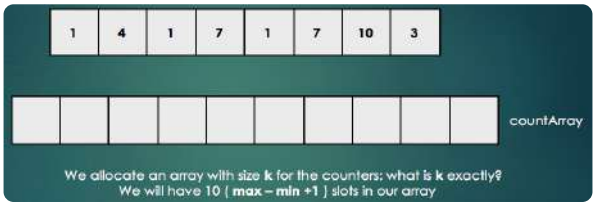
- It operates by counting the number of objects that have each distinct key value
- Integer sorting algorithm: we assume the values to be integers
- And using arithmetic on those counts to determine the positions of each key value in the output sequence
- It is only suitable for direct use in situations where the variation in keys is not significantly greater than the number of items
- It can be used as a subroutine in radix sort
- Because counting sort uses key values as indexes into an array it is not a comparison based sorting algorithm, so linearithmic running time can be reduced
- Running time: **O(N+k)**
- **N** ---> number of items we want to sort
- **k** ---> difference between the maximum and minimum key values, basically the number of possible keys
- Conclusion : it is only suitable for direct use in situations where the variation in keys is not significantly greater than the number of items



Allocate memory for an array size **k**, we want to track and count that how many occurrences are there in the original array for the given key

1. iterate through the original array **O(N)**
2. The value in the array will be the index of the temporary array: we increment the counter there
3. Traverse the array of counters (array size **k**) and print out the values **O(k)**
4. It is going to yield the numerical ordering

Watch Video:



We consider every item in our counter array! The content of the array
determines how many times the given index is present in

the original array

Content value: 3
Index: 1

Meaning: we have 3 items with key 1 in out original array

<u>Numerical ordering:</u> 1, 1, 1, 3, 4, 7, 7, 10

Overall running time: **O(k) + O(N) = O(N+k)**
Problem: **k** can be very very large, and the counting sort algorithm will be slow

<u>Pseudocode:</u>

-

```
countingSort(array, max, min)

    countArray = new array with size [max-min+1]

    for i in array
        increment countArray[i-min]
    end

    z = 0

    for i in array
        while countArray[i-min] > 0
            array[z] = i
            z = z + 1
            countArray[i-min] = countArray[i-min] - 1
        end
    end
end
```

Counting sort is not in place, we do need some additional memory
max-min+1 going to determine the size of the array
Thats why counting sort can get slow if this countArray is huge

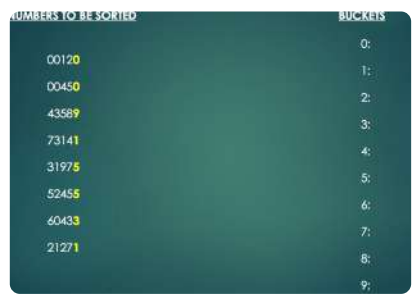
We consider every key in the original array in **O(N)** and increment the counterArray according to the right position

This operation has O(k) time complexity

Basically while the counterArray is not 0 at a given position we have to consider that index as many times as the value in counterArray[index]

It yields the numerical ordering

Radix sort

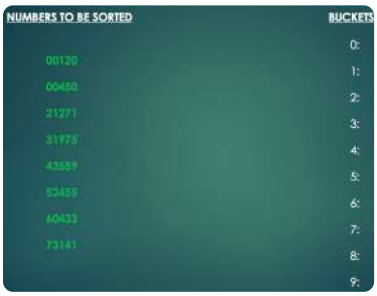


- Can be very efficient, because there are no comparisons
- So **O(N)** running time can be reached
- Running time: **O(k*N)** where **k** is the number of digits in the input number
- We sort the elements according to individual characters
- It is a stable sorting algorithm

LSD string sorting

- Least-significant-digit-first string sorting
- Consider characters from right to left
- We can use it to fixed length strings or fixed length numbers for example integers
- Sort the characters at the last column ... then keep going left and sort the columns independently
- Typical interview question: how to sort one million 32-bit integers

MSD string sort



- Most-significant-digit-first string sorting
- Consider characters from left to right
- It is sensitive to **ASCII** and **Unicode** representations
- It has several advantages
- **MSD** examines just enough characters to sort the key
- **CAN BE SUB-LINEAR IN INPUT SIZE !!!**
- **MSD** access memory randomly ... not so efficient
- Solution: we should combine it with quicksort ... this is the 3-way radix quicksort

Watch Video :

QuickSort :

- It is an efficient sorting algorithm
- It was developed by Tony Hoare in 1959 // the same person who invented quickselect algorithm
- A well implemented quicksort can outperforms heapsort and merge sort the main competitors of quicksort
- A comparison based algorithm --> not able to be faster than linearithmic time complexity
- The efficient implementation of quicksort is **NOT** stable does not keep the relative order of items with equal value
- It is in-place ---> does not need any additional memory
- On average case it has **O(N logN)** running time
- But the worst case running time is quadratic **O(N)** !!!
- It is widely used in programming languages
    - Primitive types usually quicksort is used
    - Reference types / objects ---> usually mergesort is used

It is a **divide and conquer** algorithm

- pick an element from the array: this is the pivot item
- partition phase: rearrange the array so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it // equal values can go either way

- recursively apply these steps on the subarrays

BASE CASE FOR RECURSION: arrays of size zero or one never need to be sorted

Choosing the **pivot item**

1. It is very important if we keep choosing bad pivots, the running time will be quadratic

a) we can choose a pivot at random // usually it is working fine

b) choose the middle index of the array as the pivot

Pseudocode

```
quicksort(array,low,high)

    if low >=  high  return    //This is the base case for recursion: to track when to terminate our algoithm !!!

    pivot = partition(array,low,high)          //We have to find a pivot item  we are going to partition the
                                                 array according to this item !!!

    quicksort(array,low,pivot-1)  //We call the sort method recursively on the left subarray

    quicksort(array,pivot+1,high)  //We call the sort method recursively on the right subarray

end

partition(array,low,high)

    pivotIndex = (low+high) / 2          //We should generate a random index for the pivot … OR we can
                                           have the middle   item of the array as the pivot !!!

    swap(pivotIndex,high)          //We make the pivot to be the last item in the array !!!

    i = low

    for j=low to high                    //This loop will decide what are the items that will be on the left side
        if array[j] <= array[high]                of the pivot or the right side of the pivot !!!
            swap(i,j)
            i++

    swap (i,high)          //Variable i tracks the threshold item  separates items that are smaller / greater
                             than the pivot !!!

    return i               //We return the index of the pivot so the threshold index !!!

end
```

Watch Video:

Worst case

The most naive pivot selection algorithm is to choose the first element as the pivot ~ this results in worst case behavior if the data is already sorted (the first element will always be the min)

O(N^2)

Shell sort

- Another quadratic running time sorting algorithm
- It is the generalization of the insertion sort
- Insertion sort ---> sometimes we have to make lots of shifts / swaps
- This feature is not so good thats why shell sort came to be as an enhanced insertion sort
- The method starts by sorting pairs of elements far apart from each other
- Then progressively reducing the gap between elements to be compared
- Starting with far apart elements can move some out-of-place elements into position faster than a simple nearest neighbor exchange
- Shell sort is heavily dependent on the gap sequence it uses
- Consider every **h**-th element in the array
- Such a subarray is said to be **h**-sorted
- We use insertion sort as a subprocedure the only difference is that we start sorting items far away from each other
- This rearrangement allows elements to move long distances in the original list reducing large amounts of disorder quickly
- Unstable --->it changes the relative order of elements with equal value
- Because it relies heavily on insertion sort----> it is also an adaptive algorithm so runs faster on partially sorted input

- Not so popular algorithm nowadays !!!

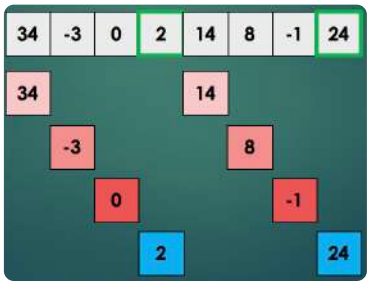
Pseudo Code:

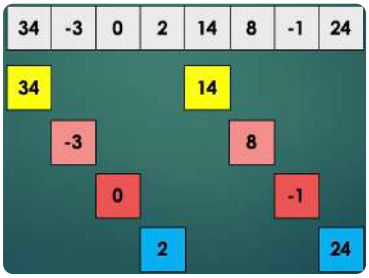
```
shellSort(array)

    for every gap in gaps      //We have to define some formula for choosing the gaps: for example at
                                 the beginning we consider every 3rd item … after that we consider every 2nd
                                 item
        for i=gap to n
            temp = array[i]

            for j=i and j >= gap and array[j-gap] > temp
                array[j] = array[j-gap]
                j = j – gap        //Basically we do a simple  insertion sort on the subarray

            end

            a[j] = temp
        end
    end
```

There are a lots of articles and models on how to choose the gaps to be the most efficient !!!
~ we start at 4 it means we consider every 4th item and sort them

34	-3	0	2	14	8	-1	24

So we have managed to found all the subarrays with gap 4, meaning all
the subarrays where the items are 4 units away from each other
~ we have to sort these sublists independently
+ insert them back into the original array





Watch Video:

RESULTS

Hash Table or Hash Map:

- Stores data with key value pairs.
- **Hash functions** accept a key and return an output unique only to that specific key.
- This is known as **hashing**, which is the concept that an input and an output have a one-to-one correspondence to map information.
- Hash functions return a unique address in memory for that data.

What you need to know:

- Designed to optimize searching, insertion, and deletion.
- **Hash collisions** are when a hash function returns the same output for two distinct inputs.
- All hash functions have this problem.
- This is often accommodated for by having the hash tables be very large.
- Hashes are important for associative arrays and database indexing.

Big O efficiency:

- Indexing: Hash Tables: O(1)
- Search: Hash Tables: O(1)
- Insertion: Hash Tables: O(1)

Binary Tree:

- Is a tree like data structure where every node has at most two children.
- There is one left and right child node.

What you need to know:

- Designed to optimize searching and sorting.
- A **degenerate tree** is an unbalanced tree, which if entirely one-sided is a essentially a linked list.
- They are comparably simple to implement than other data structures.
- Used to make **binary search trees**.
- A binary tree that uses comparable keys to assign which direction a child is.
- Left child has a key smaller than it's parent node.
- Right child has a key greater than it's parent node.
- There can be no duplicate node.
- Because of the above it is more likely to be used as a data structure than a binary tree.

Big O efficiency:

- Indexing: Binary Search Tree: O(log n)
- Search: Binary Search Tree: O(log n)
- Insertion: Binary Search Tree: O(log n)

Search Basics

Breadth First Search:

- An algorithm that searches a tree (or graph) by searching levels of the tree first, starting at the root.
- It finds every node on the same level, most often moving left to right.
- While doing this it tracks the children nodes of the nodes on the current level.
- When finished examining a level it moves to the left most node on the next level.
- The bottom-right most node is evaluated last (the node that is deepest and is farthest right of it's level).

What you need to know:

- Optimal for searching a tree that is wider than it is deep.
- Uses a queue to store information about the tree while it traverses a tree.
- Because it uses a queue it is more memory intensive than **depth first search**.
- The queue uses more memory because it needs to stores pointers

Big O efficiency:

- Search: Breadth First Search: O(|E| + |V|)
- E is number of edges
- V is number of vertices

Depth First Search:

- An algorithm that searches a tree (or graph) by searching depth of the tree first, starting at the root.
- It traverses left down a tree until it cannot go further.
- Once it reaches the end of a branch it traverses back up trying the right child of nodes on that branch, and if possible left from the right children.
- When finished examining a branch it moves to the node right of the root then tries to go left on all it's children until it reaches the bottom.
- The right most node is evaluated last (the node that is right of all it's ancestors).

What you need to know:

- Optimal for searching a tree that is deeper than it is wide.
- Uses a stack to push nodes onto.
- Because a stack is LIFO it does not need to keep track of the nodes pointers and is therefore less memory intensive than breadth first search.
- Once it cannot go further left it begins evaluating the stack.

Big O efficiency:

- Search: Depth First Search: O(|E| + |V|)
- E is number of edges
- V is number of vertices

Breadth First Search Vs. Depth First Search

- The simple answer to this question is that it depends on the size and shape of the tree.
- For wide, shallow trees use Breadth First Search
- For deep, narrow trees use Depth First Search

Nuances:

- Because BFS uses queues to store information about the nodes and its children, it could use more memory than is available on your computer. (But you probably won't have to worry about this.)
- If using a DFS on a tree that is very deep you might go unnecessarily deep in the search. See xkcd for more information.
- Breadth First Search tends to be a looping algorithm.
- Depth First Search tends to be a recursive algorithm.

Efficient Sorting Basics

Merge Sort:

- A comparison based sorting algorithm
- Divides entire dataset into groups of at most two.
- Compares each number one at a time, moving the smallest number to left of the pair.
- Once all pairs sorted it then compares left most elements of the two leftmost pairs creating a sorted group of four with the smallest numbers on the left and the largest ones on the right.
- This process is repeated until there is only one set.

What you need to know:

- This is one of the most basic sorting algorithms.
- Know that it divides all the data into as small possible sets then compares them.

Big O efficiency:

- Best Case Sort: Merge Sort: O(n)
- Average Case Sort: Merge Sort: O(n log n)
- Worst Case Sort: Merge Sort: O(n log n)

Quicksort:

- A comparison based sorting algorithm
  - Divides entire dataset in half by selecting the average element and putting all smaller elements to the left of the average.
  - It repeats this process on the left side until it is comparing only two elements at which point the left side is sorted.
  - When the left side is finished sorting it performs the same operation on the right side.
- Computer architecture favors the quicksort process.

What you need to know:

- While it has the same Big O as (or worse in some cases) many other sorting algorithms it is often faster in practice than many other sorting algorithms, such as merge sort.
- Know that it halves the data set by the average continuously until all the information is sorted.

Big O efficiency:

- Best Case Sort: Merge Sort: O(n)
- Average Case Sort: Merge Sort: O(n log n)
- Worst Case Sort: Merge Sort: O(n^2)

Bubble Sort:

- A comparison based sorting algorithm
  - It iterates left to right comparing every couplet, moving the smaller element to the left.
  - It repeats this process until it no longer moves and element to the left.

What you need to know:

- While it is very simple to implement, it is the least efficient of these three sorting methods.
- Know that it moves one space to the right comparing two elements at a time and moving the smaller on to left.

Big O efficiency:

- Best Case Sort: Merge Sort: O(n)
- Average Case Sort: Merge Sort: O(n^2)
- Worst Case Sort: Merge Sort: O(n^2)

Merge Sort Vs. Quicksort

- Quicksort is likely faster in practice.
- Merge Sort divides the set into the smallest possible groups immediately then reconstructs the incrementally as it sorts the groupings.
- Quicksort continually divides the set by the average, until the set is recursively sorted.

Recursive Algorithms

- An algorithm that calls itself in its definition.
  - **Recursive case** a conditional statement that is used to trigger the recursion.
  - **Base case** a conditional statement that is used to break the recursion.

What you need to know:

- **Stack level too deep** and **stack overflow**.
  - If you've seen either of these from a recursive algorithm, you messed up.
  - It means that your base case was never triggered because it was faulty or the problem was so massive you ran out of RAM before reaching it.
  - Knowing whether or not you will reach a base case is integral to correctly using recursion.
  - Often used in Depth First Search

Iterative Algorithms:

- An algorithm that is called repeatedly but for a finite number of times, each time being a single iteration.
  - Often used to move incrementally through a data set.

What you need to know:

- Generally, you will see iteration as loops, for, while, and until statements.
- Think of iteration as moving one at a time through a set.
- Often used to move through an array.

Recursion Vs. Iteration:

- **T**he differences between recursion and iteration can be confusing to distinguish since both can be used to implement the other. But

know that,

- Recursion is, usually, more expressive and easier to implement.
- Iteration uses less memory.
- **Functional languages** tend to use recursion. (i.e. Haskell)
- **Imperative languages** tend to use iteration. (i.e. Ruby)
- Check out this Stack Overflow post for more info.

Pseudo Code of Moving Through an Array (this is why iteration is used for this)

```
Recursion                          | Iteration
-----------------------------------|-----------------------------------
recursive method (array, n)        | iterative method (array)
  if array[n] is not nil           |   for n from 0 to size of array
    print array[n]                 |     print(array[n])
    recursive method(array, n+1)   |
  else                             |
    exit loop                      |
```

Greedy Algorithm:

- An algorithm that, while executing, selects only the information that meets a certain criteria.
- The general five components, taken from Wikipedia:
- A candidate set, from which a solution is created.
- A selection function, which chooses the best candidate to be added to the solution.
- A feasibility function, that is used to determine if a candidate can be used to contribute to a solution.
- An objective function, which assigns a value to a solution, or a partial solution.
- A solution function, which will indicate when we have discovered a complete solution.

What you need to know:

- Used to find the optimal solution for a given problem.
- Generally used on sets of data where only a small proportion of the information evaluated meets the desired result.
- Often a greedy algorithm can help reduce the Big O of an algorithm.

Pseudo Code of a Greedy Algorithm to Find Largest Difference of any Two Numbers in an Array.

```
greedy algorithm (array)
  var largest difference = 0
  var new difference = find next difference (array[n], array[n+1])
  largest difference = new difference if new difference is > largest difference
  repeat above two steps until all differences have been found
  return largest difference
```

This algorithm never needed to compare all the differences to one another, saving it an entire iteration.

Graph Algorithm:

Shortest path problem: finding a path between two vertices in a graph such that the sum of the weights of its edges is minimized

- Dijkstra algorithm
- Bellman-Ford algorithm
- A* search
- Floyd-Warshall algorithm

Dijkstra algorithm

- It was constructed by computer scientist Edsger Dijkstra in 1956
- Dijkstra can handle positive edge weights !!! // Bellman-Ford algorithm can have negative weights as well
- Several variants: it can find the shortest path from A to B, but it is able to construct a shortest path tree as well --> defines the shortest paths from a source to all the other nodes
- This is asymptotically the fastest known single-source shortest-path algorithm for arbitrary directed graphs with unbounded non-negative weights
- Dijkstra's algorithm time complexity: **O(V*logV + E)**
- Dijkstra's algorithm is a greedy one: it tries to find the global optimum with the help of local minimum--> it turns out to be good !!!
- It is greedy on every iteration we want to find the minimum distance to the next vertex possible --> appropriate data structures: heaps (binary or Fibonacci) or in general a priority queue

Pseudocode :

```
class Node
    name
    min_distance
    Node predecessor

function DijkstraAlgorithm(Graph, source)

    distance[source] = 0                    Initialization phase: distance from source is 0, because
                                            that is the starting point. All the other nodes distances are
                                            infinity because we do not know the distances in advance
    create vertex queue Q

    for v in Graph
        distance[v] = inf
        predecessor[v] = undefined  // previous node in the shortest path
        add v to Q

    while Q not empty
        u = vertex in Q with min distance // this is why to use heaps !!!
        remove v from Q

        for each neighbor v of u
            tempDist = distance[u] + distBetween(u,v)
            if tempDist < distance[v]
                distance[v] = tempDist
                predecessor[v] = u

    return distance[]  // contains the shortest distances from source to other nodes
```

Watch Video:

Spanning Trees

Kruskal algorithm

- A spanning tree of an undirected **G** graph is a subgraph that includes all the vertices of **G**
- In general, a tree may have several spanning trees
- We can assign a weight to each edge
- A minimum spanning tree is then a spanning tree with weight less than or equal to the weight of every other spanning tree
- Has lots of applications: in big data analysis, clustering algorithms, finding minimum cost for a telecommunications company laying cable to a new neighborhood
- Standard algorithms: Prim's-Jarnik, Kruskal greedy algorithms

- Worst case running time: **O(E*logE)**, so we can use it for huge graphs too
- If the edges are sorted: the algorithm will be quasi-linear
- If we multiply the weights with a constant or add a constant to the edge weights: the result will be the same
- // in physics, an invariant is a property of the system that remains unchanged under some transformation.
- In Kruskal algorithm, spanning trees are invariant under the transformation of these weights (addition, multiplication)

Prim-Jarnik algorithm

- In Kruskal implementation we build the spanning tree separately, adding the smallest edge to the spanning tree if there is no cycle
- In Prims algorithm we build the spanning tree from a given vertex, adding the smallest edge to the MST
- Kruskal edge based Prims vertex based !!!
- There are two implementations: lazy and eager
- Lazy implementation: add the new neighbour edges to the heap without deleting its content
- Eager implementation: we keep updating the heap if the distance from a vertex to the MST has changed
- Average running time: **O(E*logE)** but we need additional memory space **O(E)**
- Worst case: **O(E*logV)**

Prims VS Kruskal

- Prim's algorithm is significantly faster in the limit when you've got a really dense graph with many more edges than vertices
- Kruskal performs better in typical situations (sparse graphs) because it uses simpler data structures
- Kruskal can have better performance if the edges can be sorted in linear time or the edges are already sorted
- Prim's better if the number of edges to vertices is high (dense graphs)

Applications

- K means
- Telecommunications
- Recommendation systems

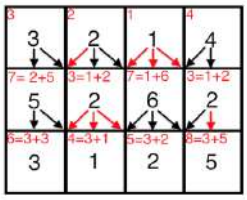
K-means clustering

- Important unsupervised machine learning technique
- We have a dataset: we want to find patterns !!!
- For examle: want to classify and cluster the points in the 2D plane
- We construct a minimal spanning tree –> leave the first k edge
- Result: we will have k clusters !!!

APPLICATIONS OF SHORTEST PATH

DAG shortest path

- If the graph is a DAG, so there is no directed cycles, it is easier to find the shortest path
- We sort the vertices into topological order: we iterate throught the topological order relaxing all edges from the actual vertex
- Topological sort algorithm computes shortest path tree in any edge weighted (can be negative!!!) DAG in time O(E+V)
- It is much faster than Bellman-Ford or Dijkstra
- Applications: solving Knapsack problem
- GPS, vehicle routing and navigation
- Detecting arbitrage situations in FX
- RIP „Routing Information Protocol”
- This is a distributed algorithm
    - Each node calculates the distances between itself and all other nodes and stores this information as a table
    - Each node sends its table to all adjacent nodes
    - When a node receives distance tables from its neighbors, it calculates the shortest routes to all other nodes and updates its own table to reflect any changes



Avidan-Shamir method

- When we want to shrink an image for example in the browser or on a smartphone without distortion
- We want to make sure the image will not deform
- We have to eliminate the least significant bit strings
- We set up an „energy function”: and remove the connected string of pixels containing the least energy
- Photoshop, GIMP use it
- We build a huge graph: vertices are the pixels and the edges are pointing from every vertex to its downward 3 neighbours
- The energy function determines what the edge weights will be
- It’s acyclic: we can use topological order shortest path to find the string of pixels to be removed

Longest path problem

- Problem of finding a simple path of maximum length in a given graph
- No polynomial time algorithm !!!
- It is an NP-hard problem
- It has a linear time solution for directed acyclic graphs (DAG) which has important applications in finding the critical path in scheduling problems
- Problem of finding a simple path of maximum length in a given graph
- No polynomial time algorithm !!! NP-hard problem
- It has a linear time solution for directed acyclic graphs (DAG) which has important applications in finding the critical path in scheduling problems
- We just have to negate the edge weights and run shortest path algorithm
- We have to use Bellman-Ford algorithm because negative edges can occur
- Application: Parallel job scheduling problem
- Given a set of jobs with durations and precedence constraints, schedule the jobs - by finding a start time to each - so as to achive the minimum completion time, while respecting the constraints

CPM: critical path method

- The method was first used between 1940 and 1943 in the Manhattan project
- Problem formulation: we want an algorithm for scheduling a set of project activities so that the total running time will be as minimal as possible
- <u>The algorithm needs</u>
- A list of all activities required to complete the project

- The time (duration) that each activity will take to complete
- The dependencies between the activities
- We create an edge weighted DAG
- Add edges with 0 weight for each precedence constraint
- We have to find the longest path in order to solve the problem
- There are no cycles in such a graph

TRIE

Motivation:

- hash-maps are very efficient so far: we can achieve **O(1)** running time for the most important operations
- Does not support sorting + hashfunction is usually not perfect: we would like to construct a data structure where search and insert operations have running time proportional to the length of the key !!!
    - ~ hashmap worst case search: **O(N)**
- we would like to get rid of collisions: this can be solved with tries + add another feature: sorting !!!
- Trie / radix tree / prefix tree
- It is a data structure to implement associative arrays
- The keys are usually strings
- Unlike BST no node in the tree stores the key associated with that given node its position in the tree defines the key with which it is associated
- All the descendants of a node have a common prefix of the string associated with that node, and the root is associated with the empty string
- Values are not necessarily associated with every node // usually leaf nodes only

-